

UNIwersYTET RZESZOWSKI
WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH
INSTYTUT INFORMATYKI



Łukasz Warzocha
134983

Informatyka

System zarządzania kliniką weterynaryjną

Praca projektowa

Praca wykonana pod kierunkiem
mgr inż. Ewa Żesławska

Rzeszów 2026

Spis treści

1. Streszczenie	6
2. Opis założeń projektu	7
2.1. Cel projektu	7
2.2. Realizacja projektu	7
2.3. Wymagania funkcjonalne oraz нефункционалне	7
3. Opis struktury projektu	10
3.1. Technologie wykorzystane w projekcie	10
3.2. Baza danych	10
3.2.1. Konfiguracja bazy danych (SQL Server)	10
3.2.2. Połączenie z bazą danych w Visual Studio	11
3.2.3. Konfiguracja środowiska Visual Studio	11
3.2.4. Normalizacja bazy oraz Integralność danych	13
3.3. Struktura klas	13
3.4. Najważniejsze metody	16
3.4.1. Logowanie	16
3.4.2. Zmiana hasła - panel użytkownika	17
3.4.3. Zmiana panelu wyświetlania - panel użytkownika	19
3.4.4. Wpłata środków - panel użytkownika	20
3.4.5. Rezerwacja zabiegu - panel użytkownika	21
3.4.6. Lista użytkowników - panel administratora	27
3.4.7. Dodawanie weterynarza - panel administratora	30
3.4.8. Dodawanie zabiegu - panel administratora	32
3.4.9. Operacje na zabiegach - panel administratora	34
3.5. Wymagania sprzętowe i narzędzia	37
4. Harmonogram realizacji projektu	38
5. Prezentacja warstwy użytkowej projektu	39
5.1. Panel logowania oraz rejestracji	39
5.2. Panel użytkownika	42
5.3. Panel administratora	59
5.4. Uruchomienie aplikacji	69
6. Podsumowanie	71
Bibliografia	73
Spis rysunków	74
Spis tabel	76
Spis listingów	77
Oświadczenie studenta o samodzielności pracy	78

1. Streszczenie

Dokumentacja aplikacji desktopowej wspomagającej zarządzanie kliniką weterynaryjną. Aplikacja m.in. umożliwia użytkownikowi na rejestrację swoich pupili (psów i kotów), rezerwację wizyt u weterynarza, zarządzanie saldem konta oraz przeglądanie historii wizyt. Administrator ma możliwość zarządzania lekarzami weterynarii, dostępnymi zabiegami oraz przeglądania i edycji wszystkich rezerwacji. Projekt został zrealizowany w języku C# z wykorzystaniem technologii WPF (interfejs graficzny) oraz ADO.NET do komunikacji z bazą danych Microsoft SQL Server. Aplikacja umożliwia m.in. rezerwację zabiegów z uwzględnieniem kalendarza weterynarza, zarządzanie pupilami, doładowanie konta użytkownika, a także dodawanie i usuwanie weterynarzy oraz zabiegów przez administratora.

Documentation of a desktop application supporting the management of a veterinary clinic. The application allows the user to register their pets (dogs and cats), book appointments with a veterinarian, manage their account balance, and browse their visit history. The administrator has the ability to manage veterinarians, available procedures, as well as view and edit all reservations. The project was developed in C# using the WPF technology (graphical user interface) and ADO.NET for communication with a Microsoft SQL Server database. The application offers features such as booking procedures with respect to the veterinarian's calendar, pet management, account top-up functionality for users, and the ability for administrators to add and remove veterinarians and procedures.

2. Opis założeń projektu

2.1. Cel projektu

Celem projektu jest stworzenie aplikacji desktopowej do zarządzania kliniką weterynaryjną. System będzie posiadał bezpieczny mechanizm logowania i rejestracji z wykorzystaniem bazy danych, a także autoryzację użytkowników na podstawie przypisanej do konta roli, takiej jak administrator lub użytkownik. Po zalogowaniu aplikacja automatycznie wyświetli odpowiedni interfejs, w zależności od roli zalogowanego użytkownika. Administrator będzie miał możliwość dodawania i usuwania weterynarzy oraz zabiegów, a także przeglądania listy wszystkich zarejestrowanych użytkowników oraz złożonych rezerwacji. Użytkownik natomiast będzie mógł doładowywać środki na swoje konto, rejestrować swoich pupili, przeglądać historię wizyt oraz rezerwować zabiegi u wybranych weterynarzy z uwzględnieniem ich kalendarza dostępności.

Przegląd istniejących rozwiązań

Na rynku dostępnych jest kilka rozwiązań o podobnym przeznaczeniu:

1. **VetApp** (<https://www.vetapp.app/>) - aplikacja dedykowana dla gabinetów weterynaryjnych. Oferuje funkcje zarządzania wizytami, kartoteką pacjentów oraz fakturowaniem. Jest rozwiązaniem płatnym, skierowanym głównie do dużych placówek.
2. **Vetster** (<https://vetster.com/>) - platforma umożliwiająca rezerwację wizyt weterynaryjnych online. Jest rozwiązaniem wymagającym opłat subskrypcyjnych.
3. **eVetPractice** (<https://evetpractice.com>) - system zarządzania praktyką weterynaryjną oferujący moduły rezerwacji i dokumentacji medycznej. Działa w modelu wiążącym się z miesięcznymi opłatami i uzależnieniem od infrastruktury dostawcy.

Przedstawione rozwiązania są zazwyczaj płatne, wymagają dostępu do internetu lub są skierowane do dużych placówek. Projektowana aplikacja stanowi prostą, bezpłatną alternatywę, dedykowaną dla małych i średnich klinik weterynaryjnych.

2.2. Realizacja projektu

Projekt został zrealizowany z wykorzystaniem języka C# oraz technologii WPF (Windows Presentation Foundation). Do przechowywania danych użyto relacyjnej bazy danych Microsoft SQL Server 2025, wraz z biblioteką ADO.NET i sterownikiem Microsoft.Data.SqlClient, która obsługuje informacje o użytkownikach, pupilach, weterynarzach, zabiegach oraz rezerwacjach. Aplikacja została opracowana w środowisku Visual Studio 2026 Insiders.

2.3. Wymagania funkcjonalne oraz нефunkcjonalne

Wymagania funkcjonalne:

1. **Logowanie i rejestracja użytkowników** - aplikacja umożliwia rejestrację nowych użytkowników poprzez formularz, w którym należy podać imię, nazwisko, adres e-mail, numer telefonu, opcjonalny

adres zamieszkania oraz hasło. Podczas logowania użytkownik wprowadza swój adres e-mail oraz hasło w celu uwierzytelnienia.

2. Panel użytkownika:

- (a) Dodawanie środków do konta - użytkownik może doładować saldo swojego konta, aby móc opłacać rezerwowane zabiegi.
- (b) Zarządzanie pupilami - użytkownik może dodawać swoich pupili (psy i koty), podając imię, gatunek, rasę, wiek oraz płeć. Może również edytować wiek pupila lub usunąć go z listy.
- (c) Rezerwacja zabiegu - użytkownik wybiera pupila, następnie zabieg, weterynarza wykonującego dany zabieg, a na końcu wolny termin z kalendarza dostępności wybranego weterynarza. System automatycznie sprawdza dostępność terminów oraz weryfikuje, czy saldo użytkownika jest wystarczające na pokrycie kosztu zabiegu.
- (d) Przeglądanie zabiegów - użytkownik ma dostęp do historii swoich rezerwacji wraz z informacjami o dacie, godzinie, weterynarzu, zabiegu oraz aktualnym statusie wizyty. Użytkownik może anulować zarezerwowany zabieg.
- (e) Zarządzanie kontem - użytkownik może edytować swoje dane osobowe oraz zmieniać hasło.

3. Panel administratora:

- (a) Zarządzanie użytkownikami - administrator ma dostęp do listy wszystkich zarejestrowanych użytkowników wraz z ich danymi i saldem konta. Może usunąć użytkownika wraz z jego pupilami i rezerwacjami.
 - (b) Zarządzanie rezerwacjami - administrator może przeglądać listę wszystkich złożonych rezerwacji z możliwością filtrowania po dacie, weterynarzu oraz statusie. Administrator może zmieniać status rezerwacji na: Zarezerwowany, Zrealizowany lub Anulowany.
 - (c) Zarządzanie weterynarzami - administrator może dodawać nowych weterynarzy, podając imię, nazwisko, specjalizację, email, telefon, hasło oraz godziny pracy w poszczególnych dniach tygodnia. Może również przypisywać zabiegi do weterynarza oraz usuwać weterynarzy z systemu.
 - (d) Zarządzanie zabiegami - administrator może dodawać nowe zabiegi do oferty kliniki, wprowadzając nazwę, czas trwania oraz cenę. Może także usuwać dostępne zabiegi.
4. Opcja wylogowania - zarówno panel użytkownika, jak i panel administratora, posiadają opcję wylogowania, która przenosi z powrotem do ekranu logowania.

Wymagania нефункционалне:

- 1. Bezpieczeństwo - dostęp do poszczególnych funkcji ograniczony zgodnie z rolą: użytkownik, administrator. Walidacja danych wejściowych na poziomie aplikacji zapobiega wprowadzaniu nieprawidłowych danych.
- 2. Skalowalność - struktura bazy danych oraz logika aplikacji umożliwiają rozszerzenie systemu o dodatkowe funkcjonalności, np. obsługę kolejnych gatunków zwierząt, powiadomienia o wizytach lub wystawianie faktur.
- 3. Responsywność - czas odpowiedzi programu na operacje użytkownika nie powinien przekraczać 1 sekundy.

4. Intuicyjność - program łatwy do użytku dzięki przejrzystej oprawie graficznej oraz podziałowi na logiczne sekcje nawigacyjne.

3. Opis struktury projektu

3.1. Technologie wykorzystane w projekcie

Interfejs użytkownika

Do budowy interfejsu graficznego zastosowałem technologię WPF (Windows Presentation Foundation) wraz z językiem znaczników XAML. WPF został wybrany ze względu na integrację z platformą .NET, bogaty zestaw wbudowanych kontrolerek oraz możliwość tworzenia nowoczesnych interfejsów bez konieczności stosowania zewnętrznych bibliotek graficznych. XAML umożliwia deklaratywne definiowanie interfejsu w sposób czytelny i całkowicie oddzielony od logiki aplikacji.

Dostęp do danych

Do komunikacji z bazą danych wykorzystałem ADO.NET ze sterownikiem Microsoft.Data.SqlClient w wersji 7.0.1. ADO.NET został wybrany ze względu na pełną kontrolę nad wykonywanymi zapytaniami SQL.

Baza danych

Jako system zarządzania bazą danych wybrałem Microsoft SQL Server 2025 ze względu na to, że używaliśmy go na labach. SQL Server oferuje obsługę zaawansowanych typów danych wykorzystanych w projekcie, takich jak TIME, DATE czy DECIMAL, a także zapewnia wydajne mechanizmy wymuszania integralności danych poprzez klucze obce i ograniczenia CHECK.

Język programowania i platforma

Aplikacja została napisana w języku C# na platformie .NET 10.0. Platforma .NET 10.0 zapewnia nowoczesne możliwości języka oraz wysoką wydajność.

Narzędzia deweloperskie

Projekt był rozwijany przy użyciu Visual Studio 2026 Insiders, zintegrowanego środowiska programistycznego firmy Microsoft, oferującego zaawansowane narzędzia do debugowania oraz zarządzania pakietami NuGet. Do zarządzania wersjami kodu użyłem systemu Git z repozytorium na platformie GitHub.

3.2. Baza danych

3.2.1. Konfiguracja bazy danych (SQL Server)

W projekcie wykorzystano relacyjną bazę danych Microsoft SQL Server. Do jej zarządzania zaleca się użycie narzędzia SQL Server Management Studio (SSMS), przewodnik do instalacji (SSMS) można znaleźć pod adresem: <https://learn.microsoft.com/en-us/ssms/install/install>. W trakcie instalacji SQL Server należy wybrać tryb uwierzytelniania Windows Authentication. Do poprawnego działania aplikacji należy:

1. Otworzyć SQL Server Management Studio i połączyć się z serwerem przy użyciu Windows Authentication.
2. W panelu Object Explorer kliknąć prawym przyciskiem myszy na Databases i wybrać opcję New Database, a następnie nadać jej nazwę **klinika_weterynaryjna**.

3. Kliknąć prawym przyciskiem myszy na nowo utworzoną bazę danych i wybrać opcję New Query.
4. Wkleić zawartość pliku **schema.sql** do edytora zapytań, a następnie kliknąć przycisk Execute (F5).
5. Po poprawnym wykonaniu skryptu baza danych będzie zawierać wszystkie tabele oraz dane początkowe niezbędne do działania aplikacji.

3.2.2. Połączenie z bazą danych w Visual Studio

Aby aplikacja mogła nawiązać połączenie z bazą danych SQL Server, wymagana jest odpowiednia konfiguracja połączenia w kodzie źródłowym. Za połączenie z bazą danych odpowiada klasa Database.

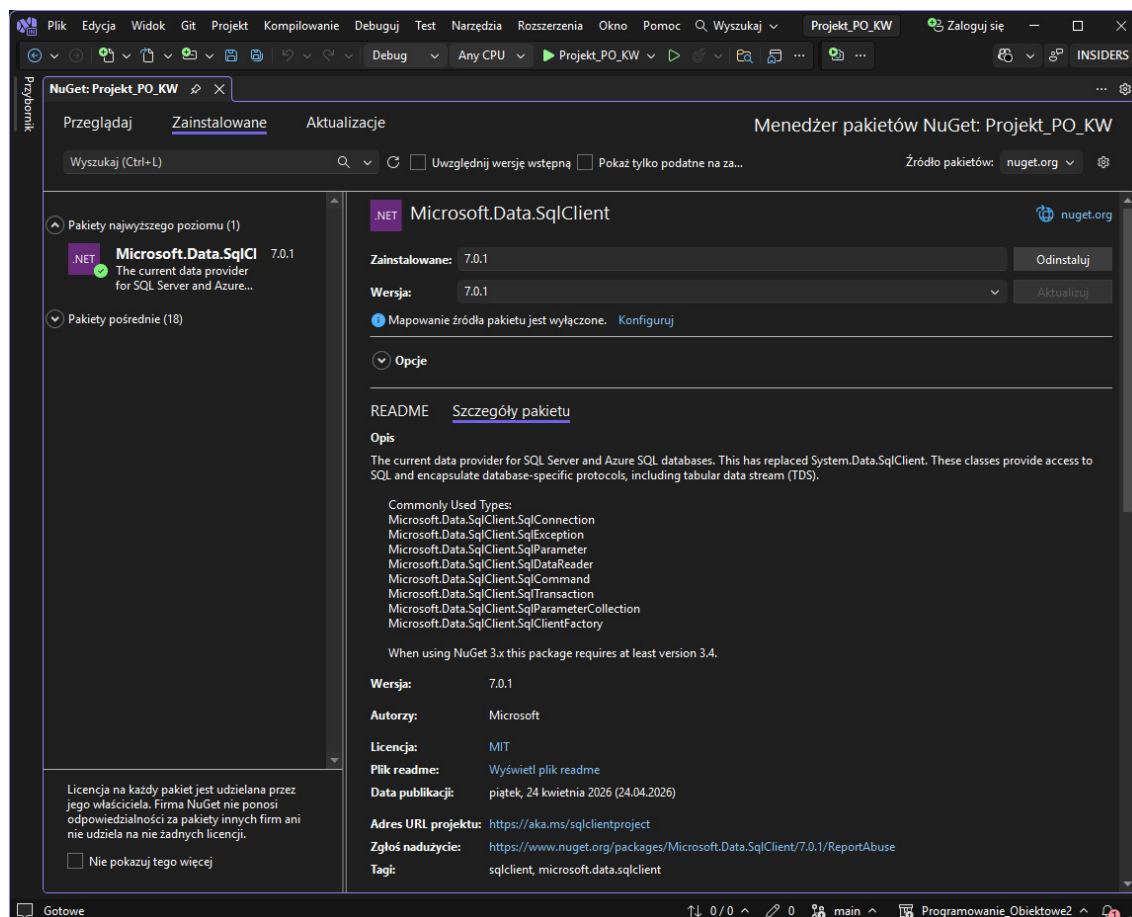
Listing 3.1. Klasa Database

```
public static class Database
{
    private static readonly string ConnectionString =
        "Server=localhost;Database=klinika_weterynaryjna;" +
        "Trusted_Connection=True;TrustServerCertificate=True;";

    public static SqlConnection GetConnection()
    {
        return new SqlConnection(ConnectionString);
    }
}
```

3.2.3. Konfiguracja środowiska Visual Studio

Do uruchomienia projektu w Visual Studio 2026 Insiders potrzebujesz platformy .NET 10.0 lub nowszej, działającego serwera SQL Server 2025 oraz sterownika Microsoft.Data.SqlClient (instalowanego przez NuGet). Konfiguracja projektu obejmuje wybór odpowiedniej wersji .NET oraz listę poprawnie załadowanych zależności NuGet.



Rys. 3.1. Sterownik Microsoft.Data.SqlClient (Menadżer pakietów NuGet).

Baza danych składa się z ośmiu tabel: Uzytkownik, Pupil, Weterynarz, Zabieg, Rezerwacja, Dostepnosc oraz tabel łączących Uzytkownik_Pupil i Weterynarz_Zabieg. Tabela Uzytkownik oraz Pupil połączone są relacją wiele do wielu za pomocą tabeli łączącej Uzytkownik_Pupil, tzn. jeden użytkownik może posiadać wielu pupili i jeden pupil może mieć wielu właścicieli. Tabela Weterynarz oraz Zabieg połączone są relacją wiele do wielu za pomocą tabeli Weterynarz_Zabieg, co oznacza, że jeden weterynarz może wykonywać wiele zabiegów, a jeden zabieg może być wykonywany przez wielu weterynarzy. Tabela Dostepnosc powiązana jest z tabelą Weterynarz relacją jeden do wielu i przechowuje informacje o godzinach pracy weterynarza w poszczególnych dniach tygodnia. Tabela Rezerwacja stanowi centralną tabelę systemu i łączy w sobie informacje o użytkowniku, pupilu, weterynarzu oraz zabiegu, przechowując datę, godzinę oraz aktualny status wizyty.

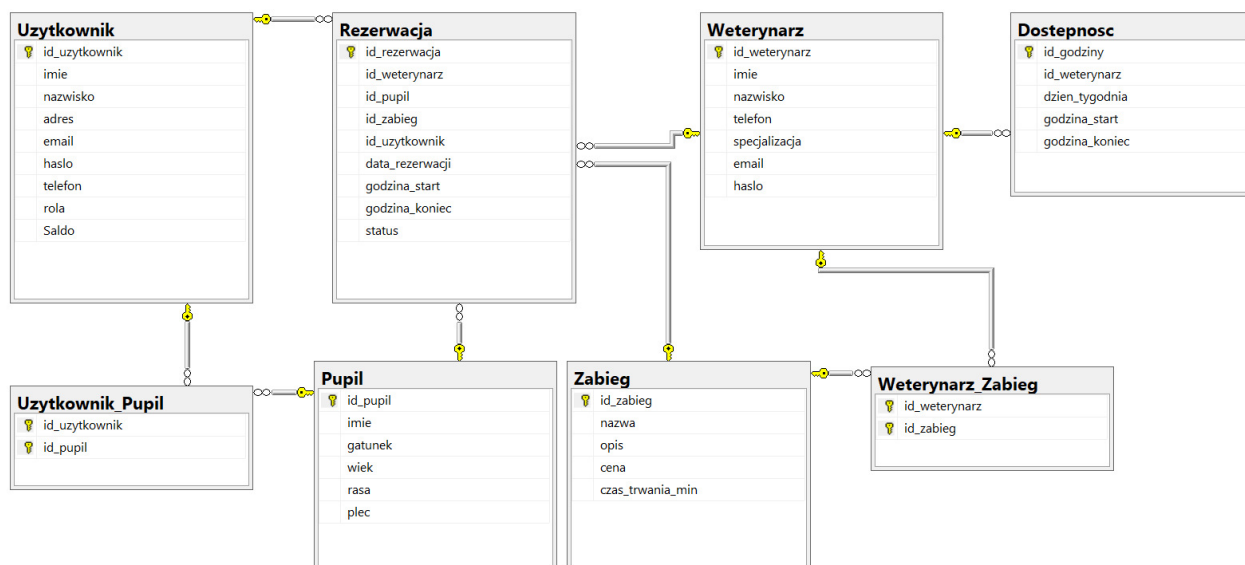


Tabela 3.1. Diagram ERD projektowanej aplikacji.

3.2.4. Normalizacja bazy oraz Integralność danych

Normalizacja bazy danych

Baza danych została zaprojektowana zgodnie z trzecią postacią normalną (3NF). Każda tabela zawiera tylko atrybuty bezpośrednio zależne od klucza głównego, eliminując redundancję danych. Relacje wiele do wielu (Uzytkownik-Pupil oraz WeterynarzZabieg) zostały rozwiązane przy użyciu dedykowanych tabel łączących.

Integralność danych

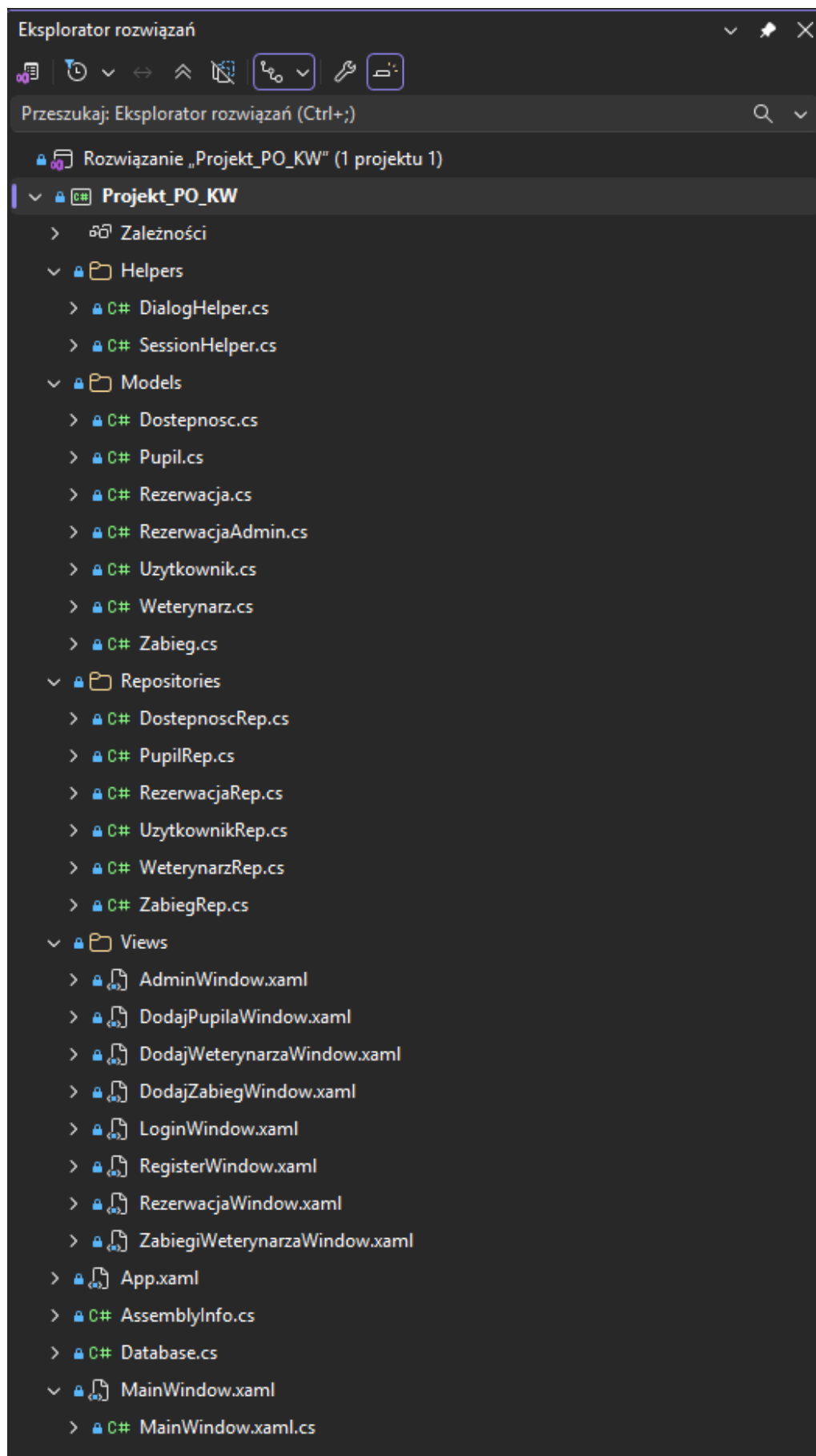
Integralność danych w bazie zapewniana jest przez następujące mechanizmy:

- **Klucze główne** - każda tabela posiada unikalny identyfikator typu INT IDENTITY, automatycznie inkrementowany przez SQL Server,
- **Klucze obce** - relacje między tabelami są wymuszane przez ograniczenia FOREIGN KEY, uniemożliwiające dodanie rekordów z nieistniejącymi odwołaniami,
- **Ograniczenia CHECK** - kolumny gatunek (Pies/Kot), plec (Samiec/Samica), rola (Uzytkownik/Administrator) oraz status (Zarezerwowany/Anulowany/Zrealizowany) posiadają ograniczenia dopuszczalnych wartości,
- **Unikalność** - kolumna email w tabeli Uzytkownik posiada ograniczenie UNIQUE, uniemożliwiające rejestrację dwóch kont na ten sam adres e-mail,
- **Walidacja po stronie aplikacji** - dodatkowa weryfikacja danych wejściowych realizowana jest w kodzie C# przed wykonaniem zapytań do bazy danych, co stanowi drugą warstwę ochrony przed nieprawidłowymi danymi.

3.3. Struktura klas

Projekt podzielony jest na cztery warstwy: Models (klasy odwzorowujące tabele bazy danych), Repositories (klasy odpowiedzialne za komunikację z bazą danych poprzez ADO.NET), Views (okna WPF) oraz

Helpers (klasy pomocnicze tzn. SessionHelper i DialogHelper). Klasa Database odpowiada za nawiązanie połączenia z bazą danych SQL Server.



Rys. 3.2. Struktura klas w projekcie.

3.4. Najważniejsze metody

3.4.1. Logowanie

Metoda `Zaloguj_sie` pobiera wartości z pól formularza: adres e-mail oraz hasło. Następnie sprawdza, czy oba pola zostały uzupełnione, jeśli nie to wyświetlany jest komunikat błędu. Jeżeli pola są wypełnione, wywoływana jest metoda `GetUser` z repozytorium `UzytkownikRep`, która wykonuje zapytanie SQL do bazy danych i zwraca obiekt użytkownika na podstawie podanego adresu e-mail oraz hasła. Jeśli użytkownik nie zostanie znaleziony w bazie danych, wyświetlany jest komunikat o nieprawidłowych danych logowania. W przypadku pomyślnego uwierzytelnienia dane zalogowanego użytkownika zapisywane są w klasie `SessionHelper`, która przechowuje je przez cały czas działania aplikacji. Następnie sprawdzana jest rola przypisana do konta, jeśli jest to administrator, otwierany jest panel administratora `AdminWindow`, w przeciwnym razie otwierany jest panel użytkownika w tym wypadku `MainWindow`. Okno logowania zostaje zamknięte. W przypadku błędu połączenia z bazą danych wyświetlany jest komunikat z informacją o przyczynie błędu.

Listing 3.2. Metoda logowania wraz z `SessionHelper` oraz metodą z zapytaniem SQL

```
//Metoda Zaloguj_sie

var email = PoleEmail.Text;
var haslo = PoleHaslo.Password;

if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(haslo))
{
    System.Windows.MessageBox.Show("Nieprawidłowy email lub hasło!", "Błąd",
    MessageBoxButton.OK, MessageBoxImage.Error);
    return;
}

try
{
    var rep = new UzytkownikRep();
    var uzytkownik = rep.GetUser(email, haslo);

    if (uzytkownik == null)
    {
        System.Windows.MessageBox.Show("Nieprawidłowy login lub hasło!", "Błąd",
        MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }

    Helpers.SessionHelper.ZalogowanyUzytkownik = uzytkownik;

    if (uzytkownik.Rola == "Administrator")
    {
        new AdminWindow().Show();
    }
    else
    {
        new MainWindow().Show();
    }
    this.Close();
}
catch (Exception ex)
{
}
```

```
System.Windows.MessageBox.Show($"Nie udało ci się połączyć z bazą danych: {ex.Message}",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

//Helper SessionHelper
public static Models.Uzytkownik? ZalogowanyUzytkownik { get; set; }

//GetUser w UzytkownikRep

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand("SELECT * FROM Uzytkownik WHERE email = @email " +
    "AND haslo = @haslo", Conn);
command.Parameters.AddWithValue("@email", email);
command.Parameters.AddWithValue("@haslo", password);

using var reader = command.ExecuteReader();
if (reader.Read())
{
    return new Uzytkownik
    {
        IdUzytkownik = (int)reader["id_uzytkownik"],
        Imie = reader["imie"].ToString()!,
        Nazwisko = reader["nazwisko"].ToString()!,
        Email = reader["email"].ToString()!,
        Telefon = reader["telefon"].ToString()!,
        Adres = reader["adres"].ToString()!,
        Haslo = reader["haslo"].ToString()!,
        Rola = reader["rola"].ToString()!,
        Saldo = (int)reader["Saldo"]!
    };
}

return null;
```

3.4.2. Zmiana hasła - panel użytkownika

Metoda `Zmiana_hasla` pobiera dane zalogowanego użytkownika z klasy `SessionHelper`. Następnie odczytuje wartości z trzech pól formularza tzn. obecne hasło, nowe hasło oraz potwierdzenie hasła. Sprawdza, czy wszystkie pola zostały uzupełnione jeśli nie, wyświetlany jest komunikat błędu. Następnie weryfikuje, czy podane obecne hasło zgadza się z hasłem przechowywanym w sesji jeśli nie, wyświetlany jest komunikat o nieprawidłowym hasle. Kolejnym krokiem jest sprawdzenie, czy nowe hasło jest identyczne z polem potwierdzającym oraz czy jego długość wynosi co najmniej 6 znaków. Przed wykonaniem zmiany użytkownik proszony jest o potwierdzenie operacji w oknie dialogowym. Następnie wywoływana jest metoda `Zmiana_hasla` z repozytorium `UzytkownikRep`, która wykonuje zapytanie `UPDATE` aktualizujące hasło w bazie danych na podstawie identyfikatora użytkownika. Po pomyślnym wykonaniu operacji pola formularza zostają wyczyszczone, a użytkownik otrzymuje komunikat o sukcesie. W przypadku błędu połączenia z bazą danych wyświetlany jest komunikat z informacją o przyczynie błędu.

Listing 3.3. Metoda zmiany hasła wraz z metodą zapytania SQL

```
// Metoda Zmien haslo w panelu uzytkownika
```

```
var uzytkownik = Helpers.SessionHelper.ZalogowanyUzytkownik;
if (uzytkownik == null) return;

var aktualne = AktualneHaslo.Password;
var nowe = NoweHaslo.Password;
var potwierdz_haslo = PowtorzHaslo.Password;

if (string.IsNullOrEmpty(aktualne) || string.IsNullOrEmpty(nowe)
|| string.IsNullOrEmpty(potwierdz_haslo))
{
    System.Windows.MessageBox.Show("Pierw musisz wypełnić wszystkie " +
    "pola dotyczące hasła!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

if (aktualne != uzytkownik.Haslo)
{
    System.Windows.MessageBox.Show("Obecne hasło jest nieprawidłowe!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

if (nowe != potwierdz_haslo)
{
    System.Windows.MessageBox.Show("Nowe hasło musi być takie same " +
    "jak w polu potwierdzającym!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

if (nowe.Length < 6)
{
    System.Windows.MessageBox.Show("Hasło musi mieć minimum 6 znaków!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

var potwierdz = System.Windows.MessageBox.Show("Czy na pewno chcesz zmienić hasło?",
"Potwierdzenie", MessageBoxButton.YesNo, MessageBoxImage.Question);
if (potwierdz == MessageBoxResult.No) return;

try
{
    uzytkownik.Haslo = nowe;
    var rep = new Repositories.UzytkownikRep();
    rep.Zmiana_hasla(uzytkownik);

    AktualneHaslo.Clear(); NoweHaslo.Clear(); PowtorzHaslo.Clear();

    System.Windows.MessageBox.Show("Twoje hasło zostało zmienione!",
    "Sukces", MessageBoxButton.OK, MessageBoxImage.Information);
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd podczas zmiany hasła: {ex.Message}",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}
```



```
}

// Zmiana_hasla w UzytkownikRep

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand("Update Uzytkownik SET haslo = " +
"@haslo WHERE id_uzytkownik = @id", Conn);
command.Parameters.AddWithValue("@haslo", user.Haslo);
command.Parameters.AddWithValue("@id", user.IdUzytkownik);

command.ExecuteNonQuery();
```

3.4.3. Zmiana panelu wyświetlania - panel użytkownika

Metoda `Nav_Click` odpowiada za nawigację między sekcjami panelu użytkownika. Na początku wszystkie panele tzn. `PanelMojeKonto`, `PanelMojePupile`, `PanelMojeWizyty` oraz `PanelDoladujSrodki` są ukrywane poprzez ustawienie właściwości `Visibility` na `Collapsed`, a znaczniki aktywności wszystkich przycisków nawigacyjnych są zerowane. Następnie kliknięty przycisk zostaje oznaczony jako aktywny poprzez ustawienie jego właściwości `Tag` na wartość `active`. W zależności od tego, który przycisk został kliknięty, wyświetlany jest odpowiedni panel. W przypadku sekcji `Moje pupile` dodatkowo wywoływana jest metoda `WczytajPupile`, która pobiera listę pupili zalogowanego użytkownika z bazy danych. Analogicznie w przypadku sekcji `Moje wizyty` wywoływana jest metoda `WczytajWizyty`, która pobiera historię rezerwacji użytkownika.

Listing 3.4. Metoda nawigacji do zmiany panelu wyświetlania

```
//Metoda NavClick

PanelMojeKonto.Visibility = Visibility.Collapsed;
PanelMojePupile.Visibility = Visibility.Collapsed;
PanelMojeWizyty.Visibility = Visibility.Collapsed;
PanelDoladujSrodki.Visibility = Visibility.Collapsed;

BtnDoladujSrodki.Tag = null;
BtnMojeKonto.Tag = null;
BtnMojePupile.Tag = null;
BtnMojeWizyty.Tag = null;

var przycisk = sender as System.Windows.Controls.Button;
przycisk!.Tag = "active";

if (przycisk == BtnMojeKonto)
    PanelMojeKonto.Visibility = Visibility.Visible;
else if (przycisk == BtnMojePupile)
{
    PanelMojePupile.Visibility = Visibility.Visible;
    WczytajPupile();
}
else if (przycisk == BtnMojeWizyty)
{
    PanelMojeWizyty.Visibility = Visibility.Visible;
    WczytajWizyty();
}
```

```
else if (przycisk == BtnDoladujSrodki)
{
    PanelDoladujSrodki.Visibility = Visibility.Visible;
}
```

3.4.4. Wpłata środków - panel użytkownika

Metoda `WplacSrodki_Click` umożliwia użytkownikowi doładowanie salda konta. Pobiera wartość wpisaną przez użytkownika w polu tekstowym i próbuje przekonwertować ją na liczbę całkowitą przy użyciu metody `int.TryParse`. Jeśli podana wartość nie jest liczbą całkowitą lub jest mniejsza bądź równa zero, wyświetlany jest komunikat błędu. W przypadku poprawnych danych użytkownik proszony jest o potwierdzenie operacji w oknie dialogowym. Po zatwierdzeniu wywoływana jest metoda `DoladujSaldo` z repozytorium `UzytkownikRep`, która wykonuje zapytanie `UPDATE` zwiększające saldo użytkownika w bazie danych o podaną kwotę. Jednocześnie saldo aktualizowane jest w obiekcie przechowywanym w `SessionHelper` oraz odświeżana jest etykieta wyświetlająca aktualne saldo w interfejsie użytkownika. Pole tekstowe zostaje wyczyszczone, a użytkownik otrzymuje komunikat o pomyślnym wykonaniu wpłaty. W przypadku błędu połączenia z bazą danych wyświetlany jest komunikat z informacją o przyczynie błędu.

Listing 3.5. Metoda wpłaty środków wraz z metodą zapytania SQL

```
// Metoda WplacSrodki_Click

var kwotaT = PoleKwota.Text;

if (!int.TryParse(kwotaT, out int kwota) || kwota <= 0)
{
    System.Windows.MessageBox.Show("Musisz podać prawidłową kwotę!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

var potwierdzenie = System.Windows.MessageBox.Show("Czy na pewno chcesz " +
    $"wpłacić {kwota} zł?",
    "Potwierdzenie", MessageBoxButton.YesNo, MessageBoxImage.Question);
if (potwierdzenie == MessageBoxResult.No) return;

try
{
    var rep = new Repositories.UzytkownikRep();
    rep.DoladujSaldo(Helpers.SessionHelper.ZalogowanyUzytkownik!.IdUzytkownik, kwota);

    Helpers.SessionHelper.ZalogowanyUzytkownik.Saldo += kwota;
    EtykietaSaldoUzytkownik.Content =
    $"Saldo: {Helpers.SessionHelper.ZalogowanyUzytkownik.Saldo} zł";
    PoleKwota.Clear();

    System.Windows.MessageBox.Show($"Wpłata kwoty {kwota} zł" +
    "przebiegła pomyślnie!",
    "Sukces", MessageBoxButton.OK, MessageBoxImage.Information);
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd: {ex.Message}",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}
```

```
// Metoda DoladujSaldo w UzytkownikRep

{
    using var Conn = Database.GetConnection();
    Conn.Open();

    var command = new SqlCommand("UPDATE Uzytkownik SET Saldo = " +
        "Saldo + @kwota WHERE id_uzytkownik = @id", Conn);
    command.Parameters.AddWithValue("@kwota", kwota);
    command.Parameters.AddWithValue("@id", idUzytkownik);
    command.ExecuteNonQuery();
}
```

3.4.5. Rezerwacja zabiegu - panel użytkownika

Metoda WybierzZabieg obsługuje wybór zabiegu przez użytkownika w pierwszym kroku rezerwacji. Na początku iteruje po wszystkich elementach listy zabiegów i resetuje kolor obramowania każdego z nich do domyślnego koloru szarego, aby usunąć poprzednie zaznaczenie. Następnie pobiera kliknięty element jako obiekt Border i zmienia kolor jego obramowania na różowy, sygnalizując aktywny wybór. Wybrany zabieg zostaje zapisany w polu _wybranyZabieg jako obiekt typu Zabieg, pobierany z kontekstu danych klikniętego elementu.

Listing 3.6. Metoda wyboru zabiegu oraz wyboru weterynarza

```
// Metoda WybierzZabieg

foreach (var item in ListaZabiegow.Items)
{
    var container =
        ListaZabiegow.ItemContainerGenerator.ContainerFromItem(item) as FrameworkElement;

    if (container != null)
    {
        var border = FindChild<Border>(container);
        if (border != null) border.BorderBrush =
            new SolidColorBrush((System.Windows.Media.Color)
                System.Windows.Media.ColorConverter.ConvertFromString("#FFE5E7EB"));
    }
}

var clicked = sender as Border;

if (clicked != null)
{
    clicked.BorderBrush = new SolidColorBrush((System.Windows.Media.Color)
        System.Windows.Media.ColorConverter.ConvertFromString("#FFFB7185"));
    _wybranyZabieg = clicked.DataContext as Models.Zabieg;
}

// Metoda WybierzWeterynarza

foreach (var item in ListaWeterynarzy.Items)
{
    var container =
```

```

ListaWeterynarzy.ItemContainerGenerator.ContainerFromItem(item) as FrameworkElement;

if (container != null)
{
    var border = FindChild<Border>(container);
    if (border != null) border.BorderBrush =
        new SolidColorBrush((System.Windows.Media.Color)
            System.Windows.Media.ColorConverter.ConvertFromString("#FFE5E7EB"));
}

var clickedBorder = sender as Border;

if (clickedBorder != null)
{
    clickedBorder.BorderBrush = new SolidColorBrush((System.Windows.Media.Color)
        System.Windows.Media.ColorConverter.ConvertFromString("#FFFB7185"));
    _wybranyWeterynarz = clickedBorder.DataContext as Models.Weterynarz;
}

```

Metoda `Dalej_Click` obsługuje nawigację między trzema krokami procesu rezerwacji wizyty. W pierwszym kroku sprawdza, czy użytkownik wybrał pupila z listy rozwijanej oraz czy zaznaczył zabieg jeśli nie, wyświetlany jest odpowiedni komunikat błędu. Następnie weryfikuje, czy saldo konta użytkownika jest wystarczające do pokrycia kosztu wybranego zabiegu. Jeśli środki są niewystarczające, wyświetlany jest komunikat informujący o aktualnym saldzie oraz koszcie zabiegu. W przypadku poprawnej walidacji pobierana jest lista weterynarzy wykonujących wybrany zabieg przy użyciu metody `GetByZabieg` z repozytorium `WeterynarzRep`, która wykonuje zapytanie `SELECT` z połączeniem tabel `Weterynarz` i `Weterynarz_Zabieg`. W drugim kroku sprawdzane jest, czy użytkownik wybrał weterynarza jeśli tak, wywoływana jest metoda `WygenerujKalendarz` i interfejs przełącza się na trzeci krok, a przycisk `Dalej` zmienia treść na `Zarezerwuj`. W trzecim a zarazem ostatnim kroku sprawdzane jest, czy użytkownik wybrał termin jeśli tak, wywoływana jest metoda `ZapiszRezerwacje`.

Listing 3.7. Metoda do rezerwacji (zachowania ciągłości między etapami) wraz z metodą zapytania SQL

```

// Metoda Dalej_Click

if (_aktualnyKrok == 1)
{
    if (WyborPupila.SelectedItem == null)
    {
        System.Windows.MessageBox.Show("Musisz wybrać pupila!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
    if (_wybranyZabieg == null)
    {
        System.Windows.MessageBox.Show("Musisz wybrać zabieg!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var saldo = Helpers.SessionHelper.ZalogowanyUzytkownik!.Saldo;
    if (saldo < (int)_wybranyZabieg.Cena)
    {

```

```
System.Windows.MessageBox.Show(
    $"Niewystarczające środki na koncie!\n" +
    $"Koszt zabiegu: {_wybranyZabieg.Cena:F2} zł\n" +
    $"Twoje saldo: {saldo} zł",
    "Brak środków", MessageBoxButton.OK, MessageBoxImage.Warning);
return;
}

_wybranyPupil = WyborPupila.SelectedItem as Models.Pupil;
var wetRep = new WeterynarzRep();
ListaWeterynarzy.ItemsSource =
wetRep.GetByZabieg(_wybranyZabieg.IdZabieg);

Krok1.Visibility = Visibility.Collapsed;
Krok2.Visibility = Visibility.Visible;
BtnWstecz.Visibility = Visibility.Visible;
_aktualnyKrok = 2;
}
else if (_aktualnyKrok == 2)
{
    if (_wybranyWeterynarz == null)
    {
        System.Windows.MessageBox.Show("Musisz wybrać weterynarza!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    WygenerujKalendarz();

    Krok2.Visibility = Visibility.Collapsed;
    Krok3.Visibility = Visibility.Visible;
    BtnDalej.Content = "Zarezerwuj";
    _aktualnyKrok = 3;
}
else if (_aktualnyKrok == 3)
{
    if (_wybranaData == default || _wybranaGodzina == default)
    {
        System.Windows.MessageBox.Show("Musisz wybrać termin!",
            "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
    ZapiszRezerwacje();
}

// Metoda GetByZabieg w klasie WeterynarzRep

var lista = new List<Weterynarz>();

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand("SELECT w.* FROM Weterynarz w " +
    "INNER JOIN Weterynarz_Zabieg wz " +
    "ON w.id_weterynarz = wz.id_weterynarz WHERE wz.id_zabieg = @id", Conn);
command.Parameters.AddWithValue("@id", idZabieg);
```

```

using var reader = command.ExecuteReader();
while (reader.Read())
{
    lista.Add(MapujWeterynarza(reader));
}

return lista;

```

Metoda WygenerujKalendarz odpowiada za dynamiczne generowanie tygodniowego widoku kalendarza dostępności wybranego weterynarza. Na początku czyszczone są wszystkie elementy siatki kalendarza. Następnie pobierana jest dostępność weterynarza z bazy danych przy użyciu metody GetByWeterynarz z repozytorium DostepnoscRep, która wykonuje zapytanie SELECT zwracające godziny pracy weterynarza w poszczególnych dniach tygodnia. Na podstawie pobranych danych generowana jest lista dni tygodnia od dnia dzisiejszego, dla których weterynarz ma zdefiniowane godziny pracy. Dla każdego dnia dodawana jest kolumna do siatki kalendarza z nagłówkiem zawierającym nazwę dnia i datę. Następnie pobierane są istniejące rezerwacje weterynarza z bazy danych. Dla każdej godziny od 8:00 do 16:00 oraz dla każdego dostępnego dnia generowany jest przycisk, którego stan zależy od trzech warunków: czy dany slot mieści się w godzinach pracy weterynarza, czy godzina zakończenia zabiegu nie przekracza końca jego pracy oraz czy w danym terminie nie istnieje już inna rezerwacja. Przyciski oznaczone są kolorami tzn. zielonym dla wolnych terminów, czerwonym dla zajętych oraz szarym dla terminów poza godzinami pracy. Po kliknięciu wolnego slotu poprzednio wybrany termin zostaje zresetowany, a nowo wybrany zostaje podświetlony na różowo z etykietą Wybrano. Wybrana data i godzina zapisywane są w odpowiednich polach.

Listing 3.8. Metoda wyświetlająca kalendarz wraz z metodą zapytania SQL

```

// Metoda WygenerujKalendarz

KalendarzGrid.Children.Clear();
KalendarzGrid.RowDefinitions.Clear();
KalendarzGrid.ColumnDefinitions.Clear();

var rep = new RezerwacjaRep();
var dostepnoscRep = new DostepnoscRep();
var dostepnosc = dostepnoscRep.GetByWeterynarz(_wybranyWeterynarz!.IdWeterynarz);

KalendarzGrid.ColumnDefinitions.Add(new ColumnDefinition {
    Width = new GridLength(60) });
var startDate = DateTime.Today;

var dni = new List<DateTime>();
for (int i = 0; i < 7; i++)
{
    var dzien = startDate.AddDays(i);
    int dzienISO = (int)dzien.DayOfWeek == 0 ? 7 : (int)dzien.DayOfWeek;
    if (dostepnosc.Any(d => d.DzienTygodnia == dzienISO)) dni.Add(dzien);
}

foreach (var dzien in dni)
    KalendarzGrid.ColumnDefinitions.Add(new ColumnDefinition {
        Width = new GridLength(1, GridUnitType.Star) });

KalendarzGrid.RowDefinitions.Add(new RowDefinition {
    Height = GridLength.Auto });

```

```
var nazwyDni = new[] { "Pon", "Wt", "Śr", "Czw", "Pt", "Sob", "Niedz" };

for (int col = 0; col < dni.Count; col++)
{
    var header = new TextBlock
    {
        Text =
            $"{nazwyDni[(int)dni[col].DayOfWeek == 0 ? 6 : (int)dni[col].DayOfWeek - 1]} {dni[col]:d.MM}",
        HorizontalAlignment = System.Windows.HorizontalAlignment.Center,
        FontWeight = FontWeights.Bold,
        Margin = new Thickness(4, 0, 4, 8)
    };

    Grid.SetRow(header, 0);
    Grid.SetColumn(header, col + 1);
    KalendarzGrid.Children.Add(header);
}

var rezerwacje = rep.GetByWeterynarz(_wybranyWeterynarz.IdWeterynarz);

for (int godz = 8; godz <= 16; godz++)
{
    var rowIndex = godz - 7;
    KalendarzGrid.RowDefinitions.Add(new RowDefinition { Height = GridLength.Auto });

    var godzLabel = new TextBlock
    {
        Text = $"{godz}:00",
        VerticalAlignment = VerticalAlignment.Center,
        Margin = new Thickness(0, 4, 8, 4),
        Foreground = new SolidColorBrush((System.Windows.Media.Color)
            System.Windows.Media.ColorConverter.ConvertFromString("#FF9CA3AF"))
    };

    Grid.SetRow(godzLabel, rowIndex);
    Grid.SetColumn(godzLabel, 0);
    KalendarzGrid.Children.Add(godzLabel);

    for (int col = 0; col < dni.Count; col++)
    {
        var dzien = dni[col];
        var godzStart =
            new TimeSpan(godz, 0, 0);
        var godzKoniec =
            godzStart.Add(TimeSpan.FromMinutes(_wybranyZabieg!.CzasTrwaniaMin));

        int dzienISO = (int)dzien.DayOfWeek == 0 ? 7 : (int)dzien.DayOfWeek;
        var dostepnoscDnia =
            dostepnosc.FirstOrDefault(d => d.DzienTygodnia == dzienISO);

        bool czyWPracy = dostepnoscDnia != null
            && godzStart >= dostepnoscDnia.GodzStart &&
            godzKoniec <= dostepnoscDnia.GodzKoniec;
        bool czyZajety = rezerwacje.Any(r =>
            r.DataRezerwacji.Date == dzien.Date && r.Godzina_Start < godzKoniec &&
            r.Godzina_Koniec > godzStart);
```

```

var button = new System.Windows.Controls.Button
{
    Content = czyZajety ? "Zajęte" : (czyWPracy ? "Wolne" : "-"),
    Margin = new Thickness(4),
    Height = 36,
    BorderThickness = new Thickness(0),
    FontWeight = FontWeights.Bold,
    FontSize = 12,
    IsEnabled = czyWPracy && !czyZajety
};

button.Background = czyZajety
? new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FFEE2E2"))
: czyWPracy
? new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FFE6F4EA"))
: new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FFF3F4F6"));

button.Foreground = czyZajety
? new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FFE53E3E"))
: czyWPracy
? new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FF2E7D32"))
: new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FF9CA3AF"));

var template = new ControlTemplate(typeof(System.Windows.Controls.Button));
var border = new FrameworkElementFactory(typeof(Border));

border.SetValue(Border.CornerRadiusProperty, new CornerRadius(8));
border.SetValue(Border.BackgroundProperty,
new TemplateBindingExtension(System.Windows.Controls.Button.BackgroundProperty));

var presenter = new FrameworkElementFactory(typeof(ContentPresenter));

presenter.SetValue(ContentPresenter.HorizontalAlignmentProperty,
System.Windows.HorizontalAlignment.Center);
presenter.SetValue(ContentPresenter.VerticalAlignmentProperty,
VerticalAlignment.Center);
border.AppendChild(presenter);
template.VisualTree = border;
button.Template = template;

var capturedDzien = dzien;
var capturedGodz = godzStart;
button.Click += (s, e) =>
{
    if (_wybranySlot != null)
    {
        _wybranySlot.Background = new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FFE6F4EA"));
        _wybranySlot.Foreground = new SolidColorBrush((System.Windows.Media.Color)
System.Windows.Media.ColorConverter.ConvertFromString("#FF2E7D32"));
        _wybranySlot.Content = "Wolne";
    }
}

```



```

    }

    var klikniety = s as System.Windows.Controls.Button;
    klikniety!.Background = new SolidColorBrush((System.Windows.Media.Color)
    System.Windows.Media.ColorConverter.ConvertFromString("#FFFB7185"));
    klikniety.Foreground = System.Windows.Media.Brushes.White;
    klikniety.Content = "Wybrano";
    _wybranySlot = klikniety;
    _wybranaData = capturedDzien;
    _wybranaGodzina = capturedGodz;
};

Grid.SetRow(button, rowIndex);
Grid.SetColumn(button, col + 1);
KalendarzGrid.Children.Add(button);
}
}

// Metoda GetByWeterynarz w DostepnoscRep

var lista = new List<Dostepnosc>();

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand("SELECT * FROM Dostepnosc" +
"WHERE id_weterynarz = @id", Conn);
command.Parameters.AddWithValue("@id", idWeterynarz);

using var reader = command.ExecuteReader();
while (reader.Read())
{
    lista.Add(new Dostepnosc
    {
        IdGodziny = (int)reader["id_godziny"],
        IdWeterynarz = (int)reader["id_weterynarz"],
        DzienTygodnia = (int)reader["dzien_tygodnia"],
        GodzStart = (TimeSpan)reader["godzina_start"],
        GodzKoniec = (TimeSpan)reader["godzina_koniec"]
    });
}

return lista;

```

3.4.6. Lista użytkowników - panel administratora

Metoda WczytajUzytkownikow pobiera listę wszystkich zarejestrowanych użytkowników z bazy danych przy użyciu metody GetAllUzytkownicy z repozytorium UzytkownikRep i przypisuje ją jako źródło danych do listy użytkowników w interfejsie. Metoda GetAllUzytkownicy wykonuje zapytanie SELECT pobierające wszystkich użytkowników z rolą Uzytkownik, posortowanych według nazwiska.

Metoda UsunUzytkownika_Click obsługuje usunięcie użytkownika przez administratora. Pobiera identyfikator użytkownika z właściwości Tag klikniętego przycisku i wyświetla okno dialogowe z prośbą o potwierdzenie operacji, informując że usunięcie użytkownika spowoduje również usunięcie jego pupili i rezerwacji. Po potwierdzeniu wywoływana jest metoda UsunUzytkownika z repozytorium UzytkownikRep

i lista użytkowników jest odświeżana. Metoda `UsunUzytkownika` wykonuje operację usunięcia w kilku krokach: najpierw usuwa wszystkie rezerwacje użytkownika, następnie pobiera listę jego pupili, usuwa powiązania użytkownika z pupilami z tabeli `Uzytkownik_Pupil`, a następnie dla każdego pupila sprawdza, czy nie posiada już innych właścicieli jeśli nie, pupil zostaje trwale usunięty z tabeli `Pupil`. Na końcu usuwany jest sam rekord użytkownika z tabeli `Uzytkownik`.

Listing 3.9. Metody usuwania i wczytywania użytkowników wraz z metodą zapytania SQL

```
// WczytajUzytkownikow

try
{
    var rep = new UzytkownikRep();
    ListaUzytkownikow.ItemsSource = rep.GetAllUzytkownicy();
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd: {ex.Message}",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

// Metoda GetAllUzytkownicy w UzytkownikRep

var lista = new List<Uzytkownik>();

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand(
    "SELECT * FROM Uzytkownik WHERE rola = 'Uzytkownik' " +
    "ORDER BY nazwisko", Conn);
using var reader = command.ExecuteReader();

while (reader.Read())
{
    lista.Add(new Uzytkownik
    {
        IdUzytkownik = (int)reader["id_uzytkownik"],
        Imie = reader["imie"].ToString()!,
        Nazwisko = reader["nazwisko"].ToString()!,
        Email = reader["email"].ToString()!,
        Telefon = reader["telefon"].ToString()!,
        Saldo = (int)reader["Saldo"]
    });
}

// return lista;

// Metoda UsunUzytkownika_Click

var button = sender as System.Windows.Controls.Button;
var id = (int)button!.Tag;

var potwierdzenie = System.Windows.MessageBox.Show("Usunięcie użytkownika usunie też " +
"jego pupile i rezerwacje. Kontynuować?",
```

```
"Potwierdzenie", MessageBoxButton.YesNo, MessageBoxImage.Warning);
if (potwierdzenie == MessageBoxResult.No) return;

try
{
    var rep = new UzytkownikRep();
    rep.UsunUzytkownika(id);
    WczytajUzytkownikow();
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd usuwania: {ex.Message}",
        "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

// Metoda UsunUzytkownika w UzytkownikRep

using var Conn = Database.GetConnection();
Conn.Open();

var coomandRez = new SqlCommand("DELETE FROM Rezerwacja" +
    "WHERE id_uzytkownik = @id", Conn);
coomandRez.Parameters.AddWithValue("@id", idUzytkownik);
coomandRez.ExecuteNonQuery();

var coomandPupile = new SqlCommand("SELECT id_pupil FROM" +
    "Uzytkownik_Pupil WHERE id_uzytkownik = @id", Conn);
coomandPupile.Parameters.AddWithValue("@id", idUzytkownik);
var idPupili = new List<int>();
using (var reader = coomandPupile.ExecuteReader())
    while (reader.Read()) idPupili.Add((int)reader["id_pupil"]);

var coomandUP = new SqlCommand("DELETE FROM Uzytkownik_Pupil" +
    "WHERE id_uzytkownik = @id", Conn);
coomandUP.Parameters.AddWithValue("@id", idUzytkownik);
coomandUP.ExecuteNonQuery();

foreach (var idPupil in idPupili)
{
    var coomandCheck = new SqlCommand("SELECT COUNT(*) FROM" +
        "Uzytkownik_Pupil WHERE id_pupil = @id", Conn);
    coomandCheck.Parameters.AddWithValue("@id", idPupil);
    int count = (int)coomandCheck.ExecuteScalar();
    if (count == 0)
    {
        var coomandddDelPupil = new SqlCommand("DELETE FROM Pupil" +
            "WHERE id_pupil = @id", Conn);
        coomandddDelPupil.Parameters.AddWithValue("@id", idPupil);
        coomandddDelPupil.ExecuteNonQuery();
    }
}

var coomandUser = new SqlCommand("DELETE FROM Uzytkownik" +
    "WHERE id_uzytkownik = @id", Conn);
coomandUser.Parameters.AddWithValue("@id", idUzytkownik);
coomandUser.ExecuteNonQuery();
```

3.4.7. Dodawanie weterynarza - panel administratora

Metoda `DodajWeterynarza_Click` obsługuje dodawanie nowego weterynarza przez administratora. Pobiera wartości ze wszystkich pól formularza i sprawdza, czy wymagane pola (imię, nazwisko, email oraz hasło) zostały uzupełnione. Jeśli nie, wyświetlany jest komunikat błędu. W przypadku poprawnej walidacji wywoływana jest metoda `Dodaj` z repozytorium `WeterynarzRep`, która wykonuje zapytanie `INSERT` dodające nowego weterynarza do tabeli `Weterynarz` w bazie danych. Następnie pobierany jest identyfikator nowo dodanego weterynarza przy użyciu metody `GetIdByEmail`. Dla każdego dnia tygodnia od poniedziałku do piątku wywoływana jest metoda pomocnicza `ZapiszDostepnosc`, która sprawdza, czy pola godzin zostały uzupełnione oraz czy godzina rozpoczęcia jest wcześniejsza niż godzina zakończenia, jeśli warunki są spełnione, wywoływana jest metoda `Dodaj` z repozytorium `DostepnoscRep`, która wykonuje zapytanie `INSERT` zapisujące godziny pracy weterynarza w danym dniu tygodnia do tabeli `Dostepnosc`. Jeśli pola godzin dla danego dnia są puste, wpis dla tego dnia nie jest tworzony, co oznacza, że weterynarz nie pracuje w tym dniu.

Listing 3.10. Metoda dodawania weterynarza wraz z metodami zapytań SQL

```
// Metoda DodajWeterynarza_Click w DodajWeterynarzaWindow

var imie = PoleImie.Text;
var nazwisko = PoleNazwisko.Text;
var specjalizacja = PoleSpecjalizacja.Text;
var email = PoleEmail.Text;
var telefon = PoleTelefon.Text;
var haslo = PoleHaslo.Password;

if (string.IsNullOrEmpty(imie) || string.IsNullOrEmpty(nazwisko) ||
    string.IsNullOrEmpty(email) || string.IsNullOrEmpty(haslo))
{
    System.Windows.MessageBox.Show("Musisz wypełnić wymagane pola" +
        "(imię, nazwisko, email, hasło)!",
        "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

try
{
    var rep = new Repositories.WeterynarzRep();
    rep.Dodaj(new Models.Weterynarz
    {
        Imie = imie,
        Nazwisko = nazwisko,
        Specjalizacja = specjalizacja,
        Email = email,
        Telefon = telefon,
        Haslo = haslo
    });

    int idWet = rep.GetIdByEmail(email);

    var dostRep = new Repositories.DostepnoscRep();
    ZapiszDostepnosc(dostRep, idWet, 1, PonOd.Text, PonDo.Text);
    ZapiszDostepnosc(dostRep, idWet, 2, WtOd.Text, WtDo.Text);
    ZapiszDostepnosc(dostRep, idWet, 3, SrOd.Text, SrDo.Text);
    ZapiszDostepnosc(dostRep, idWet, 4, CzwOd.Text, CzwDo.Text);
}
```

```
ZapiszDostepnosc(dostRep, idWet, 5, PtOd.Text, PtDo.Text);

System.Windows.MessageBox.Show("Weterynarz został dodany pomyślnie!",
    "Sukces", MessageBoxButton.OK, MessageBoxImage.Information);
this.Close();
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd: {ex.Message}",
        "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

// Metoda Dodaj w WeterynarzRep

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand(
    "INSERT INTO Weterynarz (imie, nazwisko, telefon, specjalizacja, email, haslo) " +
    "VALUES (@imie, @nazwisko, @telefon, @specjalizacja, @email, @haslo)", Conn);
command.Parameters.AddWithValue("@imie", w.Imie);
command.Parameters.AddWithValue("@nazwisko", w.Nazwisko);
command.Parameters.AddWithValue("@telefon", w.Telefon);
command.Parameters.AddWithValue("@specjalizacja", w.Specjalizacja);
command.Parameters.AddWithValue("@email", w.Email);
command.Parameters.AddWithValue("@haslo", w.Haslo);
command.ExecuteNonQuery();

// Metoda ZapiszDostepnosc w DodajWeterynarzaWindow

if (string.IsNullOrEmpty(od) || string.IsNullOrEmpty(doo)) return;

if (!TimeSpan.TryParse(od, out var gOd)
    || !TimeSpan.TryParse(doo, out var gDo) || gOd >= gDo) return;

rep.Dodaj(new Models.Dostepnosc
{
    IdWeterynarz = idWet,
    DzieńTygodnia = dzien,
    GodzStart = gOd,
    GodzKoniec = gDo
});

// Metoda Dodaj w DostepnoscRep

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand("INSERT INTO Dostepnosc " +
    "(id_weterynarz, dzien_tygodnia, godzina_start, godzina_koniec) " +
    "VALUES (@id, @dzien, @od, @do)", Conn);
command.Parameters.AddWithValue("@id", d.IdWeterynarz);
command.Parameters.AddWithValue("@dzien", d.DzieńTygodnia);
command.Parameters.AddWithValue("@od", d.GodzStart);
command.Parameters.AddWithValue("@do", d.GodzKoniec);
command.ExecuteNonQuery();
```

3.4.8. Dodawanie zabiegu - panel administratora

Metoda `Dodaj_Click` w oknie `DodajZabiegWindow` obsługuje dodawanie nowego zabiegu przez administratora. Pobiera wartości z trzech pól formularza: nazwę zabiegu, czas trwania oraz cenę. Sprawdza, czy wszystkie pola zostały uzupełnione jeśli nie, wyświetlany jest komunikat błędu. Następnie weryfikuje, czy podany czas trwania jest poprawną dodatnią liczbą całkowitą przy użyciu metody `int.TryParse`, oraz czy cena jest poprawną nieujemną liczbą dziesiętną przy użyciu metody `decimal.TryParse`. W przypadku poprawnej walidacji wywoływana jest metoda `Dodaj` z repozytorium `ZabiegRep`, która wykonuje zapytanie `INSERT` dodające nowy zabieg do tabeli `Zabieg` w bazie danych. Pole opis jest opcjonalne jeśli nie zostało uzupełnione, do bazy zapisywana jest wartość `NULL`.

Natomiast metoda `UsunZabieg_Click` obsługuje usunięcie zabiegu. Pobiera identyfikator zabiegu z właściwości `Tag` klikniętego przycisku i wyświetla okno dialogowe z prośbą o potwierdzenie operacji, informując że usunięcie zabiegu spowoduje również usunięcie powiązanych rezerwacji. Po potwierdzeniu wywoływana jest metoda `Usun` z repozytorium `ZabiegRep`, która wykonuje operację usunięcia w trzech krokach: najpierw usuwa wszystkie rezerwacje powiązane z danym zabiegiem z tabeli `Rezerwacja`, następnie usuwa powiązania z weterynarzami z tabeli `Weterynarz_Zabieg`, a na końcu usuwa sam rekord zabiegu z tabeli `Zabieg`. Po pomyślnym usunięciu lista zabiegów jest odświeżana.

Listing 3.11. Metody dodawania i usuwania zabiegów wraz z metodami zapytań SQL

```
// Metoda Dodaj_Click w DodajZabiegWindow

var nazwa = PoleNazwa.Text;
var czasTxt = PoleCzas.Text;
var cenaTxt = PoleCena.Text;

if (string.IsNullOrEmpty(nazwa)
    || string.IsNullOrEmpty(czasTxt) || string.IsNullOrEmpty(cenaTxt))
{
    System.Windows.MessageBox.Show("Musisz wypełnić wszystkie pola!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

if (!int.TryParse(czasTxt, out int czas) || czas <= 0)
{
    System.Windows.MessageBox.Show("Musisz podać prawidłowy" +
    "czas trwania (minuty)!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

if (!decimal.TryParse(cenaTxt, out decimal cena) || cena < 0)
{
    System.Windows.MessageBox.Show("Musisz podać prawidłową cenę!",
    "Błąd", MessageBoxButton.OK, MessageBoxImage.Warning);
    return;
}

try
{
    var rep = new Repositories.ZabiegRep();
    rep.Dodaj(new Models.Zabieg
    {
```

```
Nazwa = nazwa,
CzasTrwaniaMin = czas,
Cena = cena
});

System.Windows.MessageBox.Show("Zabieg został dodany pomyślnie!",
    "Sukces", MessageBoxButton.OK, MessageBoxImage.Information);
this.Close();
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd: {ex.Message}",
        "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

// Metoda Dodaj w ZabiegRep

using var Conn = Database.GetConnection();
Conn.Open();

var coomand = new SqlCommand("INSERT INTO Zabieg " +
    "(nazwa, opis, cena, czas_trwania_min) VALUES " +
    "(@nazwa, @opis, @cena, @czas)", Conn);
coomand.Parameters.AddWithValue("@nazwa", z.Nazwa);
coomand.Parameters.AddWithValue("@opis", (object?)z.Opis ?? DBNull.Value);
coomand.Parameters.AddWithValue("@cena", z.Cena);
coomand.Parameters.AddWithValue("@czas", z.CzasTrwaniaMin);
coomand.ExecuteNonQuery();

// Metoda UsunZabieg_Click w panelu administratora

var button = sender as System.Windows.Controls.Button;
var id = (int)button!.Tag;

var potwierdzenie = System.Windows.MessageBox.Show("Usunięcie zabiegu usunie " +
    "też powiązane rezerwacje. Kontynuować?",
    "Potwierdzenie", MessageBoxButton.YesNo, MessageBoxImage.Warning);
if (potwierdzenie == MessageBoxResult.No) return;

try
{
    var rep = new ZabiegRep();
    rep.Usun(id);
    WczytajZabiegi();
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd usuwania: {ex.Message}",
        "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

// Metoda Usun w ZabiegRep

using var Conn = Database.GetConnection();
Conn.Open();
```

```

var coomandRez = new SqlCommand("DELETE FROM Rezerwacja" +
    "WHERE id_zabieg = @id", Conn);
coomandRez.Parameters.AddWithValue("@id", idZabieg);
coomandRez.ExecuteNonQuery();

var coomandWZ = new SqlCommand("DELETE FROM Weterynarz_Zabieg" +
    "WHERE id_zabieg = @id", Conn);
coomandWZ.Parameters.AddWithValue("@id", idZabieg);
coomandWZ.ExecuteNonQuery();

var coomand = new SqlCommand("DELETE FROM Zabieg" +
    "WHERE id_zabieg = @id", Conn);
coomand.Parameters.AddWithValue("@id", idZabieg);
coomand.ExecuteNonQuery();

```

3.4.9. Operacje na zabiegach - panel administratora

Metoda `WczytajRezerwacje` pobiera wszystkie rezerwacje z bazy danych przy użyciu metody `GetAllAdmin` z repozytorium `RezerwacjaRep`, która wykonuje zapytanie `SELECT` z czterokrotnym połączeniem tabel `Weterynarz`, `Pupil`, `Zabieg`, `Uzytkownik` w celu pobrania pełnych informacji o każdej rezerwacji, takich jak imię i nazwisko weterynarza, imię pupila, nazwa zabiegu oraz imię i nazwisko właściciela. Wyniki są sortowane malejąco według daty rezerwacji. Pobrana lista zapisywana jest w polu `_wszystkieRezerwacje`, a następnie wywoływana jest metoda `ZastosujFiltry`.

Metoda `ZastosujFiltry` filtruje listę wszystkich rezerwacji na podstawie trzech kryteriów: wybranej daty, wybranego weterynarza oraz wybranego statusu. Filtrowanie odbywa się sekwencyjnie tzn. każdy aktywny filtr zawęży zbiór wyników. Aby uniknąć wywoływania filtrowania podczas inicjalizacji kontrolek, metoda sprawdza flagę `_inicjalizacja`. Metoda `WyczyszcFiltry_Click` resetuje wszystkie filtry do wartości domyślnych i ponownie wywołuje `ZastosujFiltry`.

Metoda `StatusRezerwacji_Changed` obsługuje zmianę statusu rezerwacji bezpośrednio z poziomu listy. Pobiera identyfikator rezerwacji z właściwości `Tag` listy rozwijanej oraz nową wartość statusu z wybranego elementu. Następnie wywołuje metodę `ZmienStatus` z repozytorium `RezerwacjaRep`, która wykonuje zapytanie `UPDATE` aktualizujące status rezerwacji w bazie danych.

Metoda `AktualizujPrzeterminowane` wywołwana jest automatycznie przy każdym uruchomieniu aplikacji w klasie `App.xaml.cs`. Wykonuje zapytanie `UPDATE` zmieniające status wszystkich rezerwacji o statusie `Zarezerwowany`, których godzina zakończenia już minęła, na status `Zrealizowany`. Dzięki temu system automatycznie aktualizuje historię wizyt.

Listing 3.12. Metoda aktualizacji statusów oraz metody związane z zabiegami wraz z metodami zapytań SQL

```

// Metoda WczytajRezerwacje w panelu administratora

try
{
    var rep = new RezerwacjaRep();
    _wszystkieRezerwacje = rep.GetAllAdmin();
    ZastosujFiltry();
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd: {ex.Message}",
        "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

```



```
}

// Metoda GetAllAdmin w RezerwacjaRep

var lista = new List<RezerwacjaAdmin>();

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand(
    "SELECT r.id_rezerwacja, r.id_weterynarz, r.data_rezerwacji, " +
    "r.godzina_start, r.godzina_koniec, r.status, " +
    "w.imie + ' ' + w.nazwisko AS imie_nazwisko_weterynarza, " +
    "p.imie AS imie_pupila, " +
    "z.nazwa AS nazwa_zabiegu, " +
    "u.imie + ' ' + u.nazwisko AS imie_nazwisko_uzytkownika " +
    "FROM Rezerwacja r " +
    "INNER JOIN Weterynarz w ON r.id_weterynarz = w.id_weterynarz " +
    "INNER JOIN Pupil p ON r.id_pupil = p.id_pupil " +
    "INNER JOIN Zabieg z ON r.id_zabieg = z.id_zabieg " +
    "INNER JOIN Uzytkownik u ON r.id_uzytkownik = u.id_uzytkownik " +
    "ORDER BY r.data_rezerwacji DESC, r.godzina_start", Conn);

using var reader = command.ExecuteReader();
while (reader.Read())
{
    lista.Add(new RezerwacjaAdmin
    {
        IdRezerwacja = (int)reader["id_rezerwacja"],
        IdWeterynarz = (int)reader["id_weterynarz"],
        DataRezerwacji = (DateTime)reader["data_rezerwacji"],
        GodzStart = (TimeSpan)reader["godzina_start"],
        GodzKoniec = (TimeSpan)reader["godzina_koniec"],
        Status = reader["status"].ToString()!,
        ImieNazwiskoWeterynarza =
            reader["imie_nazwisko_weterynarza"].ToString()!,
        ImiePupila = reader["imie_pupila"].ToString()!,
        NazwaZabiegu = reader["nazwa_zabiegu"].ToString()!,
        ImieNazwiskoUzytkownika =
            reader["imie_nazwisko_uzytkownika"].ToString()!
    });
}

// return lista;

// Metoda Filtry_Changed w panelu administratora

if (_inicjalizacja) return;
ZastosujFiltry();

// Metoda ZastosujFiltry w panelu administratora

if (_inicjalizacja) return;
if (_wszystkieRezerwacje == null) return;
if (ListaRezerwacji == null) return;
```

```
var wynik = _wszystkieRezerwacje.AsEnumerable();

if (FiltrData.SelectedDate.HasValue)
{
    wynik =
        wynik.Where(r => r.DataRezerwacji.Date == FiltrData.SelectedDate.Value.Date);
}

if (FiltrWeterynarz.SelectedItem is Weterynarz wet && wet.IdWeterynarz != 0)
{
    wynik = wynik.Where(r => r.IdWeterynarz == wet.IdWeterynarz);
}

if (FiltrStatus.SelectedItem is ComboBoxItem statusItem
    && statusItem.Content?.ToString() != "Wszystkie statusy")
{
    wynik =
        wynik.Where(r => r.Status == statusItem.Content?.ToString());
}

ListaRezerwacji.ItemsSource = wynik.ToList();

// Metoda WyczyszcFiltry_Click

FiltrData.SelectedDate = null;
if (FiltrWeterynarz.Items.Count > 0) FiltrWeterynarz.SelectedIndex = 0;
FiltrStatus.SelectedIndex = 0;
ZastosujFiltry();

// Metoda StatusRezerwacji_Changed w panelu administratora

var combo = sender as System.Windows.Controls.ComboBox;
if (combo?.Tag == null) return;

var idRezerwacja = (int)combo.Tag;
var nowyStatus = (combo.SelectedItem as ComboBoxItem)?.Content?.ToString();
if (nowyStatus == null) return;

try
{
    var rep = new RezerwacjaRep();
    rep.ZmienStatus(idRezerwacja, nowyStatus);
}
catch (Exception ex)
{
    System.Windows.MessageBox.Show($"Błąd zmiany statusu: {ex.Message}",
        "Błąd", MessageBoxButton.OK, MessageBoxImage.Error);
}

// Metoda ZmienStatus w RezerwacjaRep

using var Conn = Database.GetConnection();
Conn.Open();

var command = new SqlCommand(
    "UPDATE Rezerwacja SET status = @status " +
    "WHERE id_rezerwacja = @id", Conn);
command.Parameters.AddWithValue("@status", nowyStatus);
```

```
command.Parameters.AddWithValue("@id", idRezerwacja);
command.ExecuteNonQuery();

// Metoda AktualizujPrzeterminowane W App.xaml.cs

try
{
    using var Conn = Database.GetConnection();
    Conn.Open();

    var command = new SqlCommand("UPDATE Rezerwacja SET status = " +
        "'Zrealizowany' WHERE status = 'Zarezerwowany' " +
        "AND CAST(data_rezerwacji AS DATETIME) + CAST(godzina_koniec AS DATETIME) " +
        "< GETDATE()", Conn);
    command.ExecuteNonQuery();
}
catch { }
```

3.5. Wymagania sprzętowe i narzędzia

- **Procesor:** minimum 2-rdzeniowy, 2.0 GHz lub szybszy.
- **Pamięć RAM:** minimum 8 GB.
- **System operacyjny:** Windows 10 (64-bit) lub nowszy.
- **.NET:** wersja 10.0 lub nowsza.
- **Microsoft SQL Server:** wersja 2025 (wersja używana w projekcie).
- **Visual Studio Insiders:** wersja 2026 (wersja używana w projekcie).
- **Sterownik ADO.NET:** Microsoft.Data.SqlClient w wersji 7.0.1 (instalowany przez NuGet).

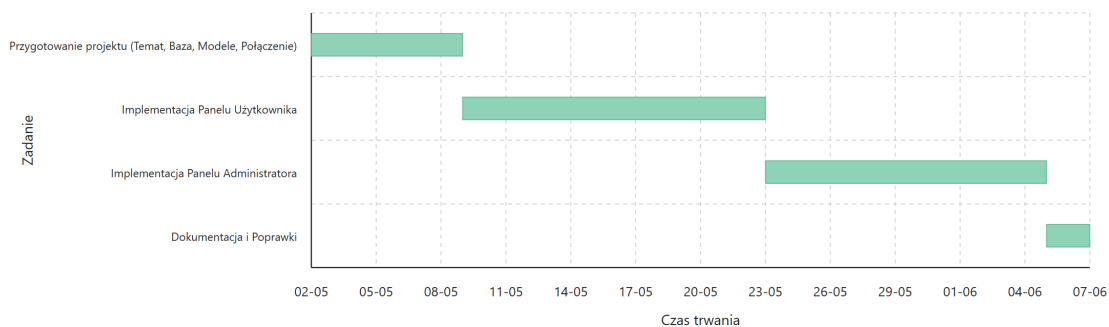
Narzędzia wykorzystane w projekcie

- **Visual Studio Insiders 2026** - środowisko programistyczne do tworzenia aplikacji w języku C#,
- **Git** - system kontroli wersji,
- **WPF (Windows Presentation Foundation)** - technologia do tworzenia graficznego interfejsu użytkownika,
- **Microsoft SQL Server 2025** - system zarządzania relacyjną bazą danych,
- **SQL Server Management Studio (SSMS)** - narzędzie do zarządzania bazą danych SQL Server,
- **Microsoft.Data.SqlClient 7.0.1** - sterownik ADO.NET do komunikacji z bazą danych SQL Server,
- **Strona online-convert.com** - strona wykorzystana do konwersji grafik do formatu EPS.

4. Harmonogram realizacji projektu

W realizacji projektu użyłem Git jako systemu kontroli wersji, a repozytorium jest zapisane na platformie GitHub pod adresem: https://github.com/lukaszwarzocha/Programowanie_Objektowe2.git.

Realizację projektu przedstawiam na diagramie Gantta. Prace rozpocząłem od przygotowania projektu, które obejmowało m.in. zaprojektowanie bazy danych, stworzenie modeli oraz konfigurację połączenia z bazą danych. Następnie przystąpiłem do tworzenia interfejsu graficznego w technologii WPF równolegle tworząc implementację panelu użytkownika. Kolejnym etapem była implementacja panelu administratora. Ostatnim etapem było tworzenie dokumentacji do projektu.

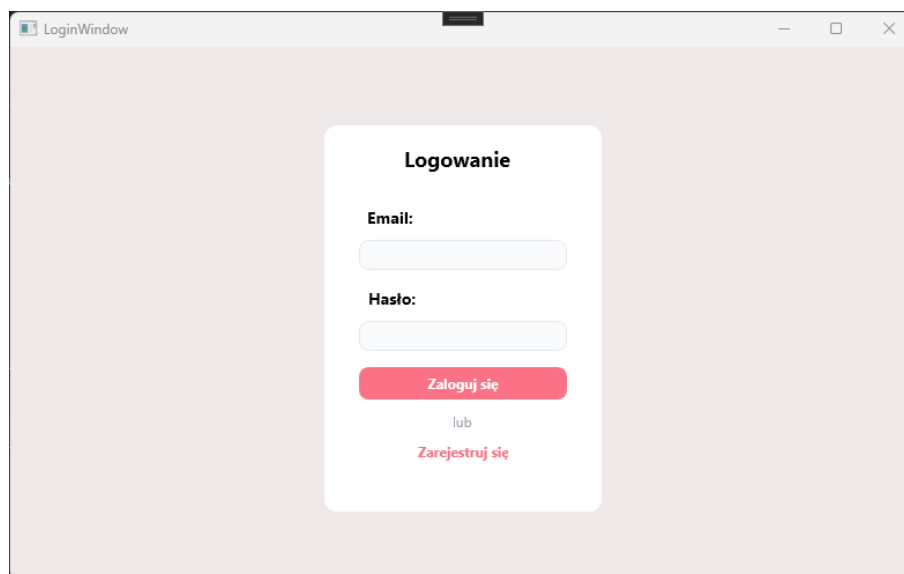


Rys. 4.1. Diagram Gantta.

5. Prezentacja warstwy użytkowej projektu

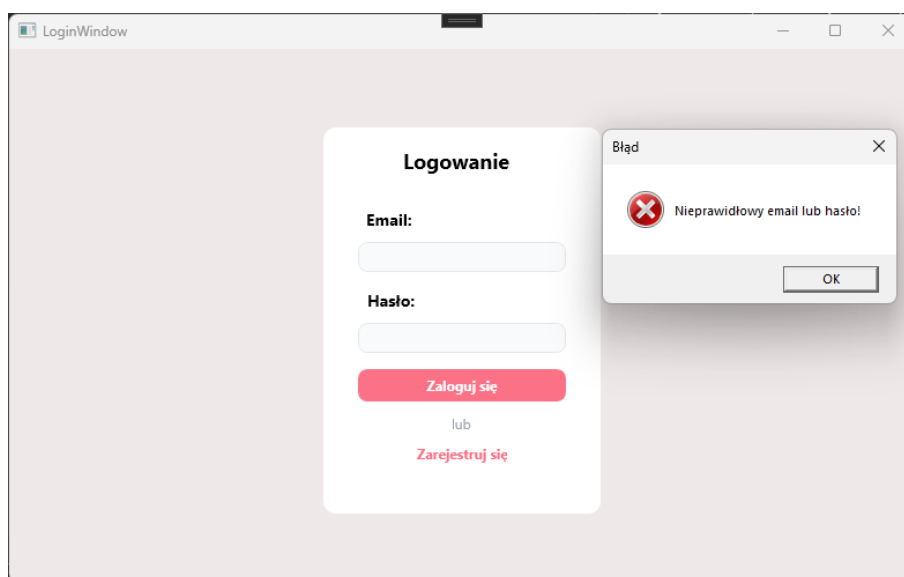
5.1. Panel logowania oraz rejestracji

Panel logowania umożliwia użytkownikowi uwierzytelnienie się w systemie. Formularz zawiera dwa pola: adres e-mail oraz hasło. Po kliknięciu przycisku Zaloguj się aplikacja weryfikuje poprawność wprowadzonych danych. Po poprawnym zalogowaniu użytkownik zostaje przekierowany do panelu użytkownika lub panelu administratora, w zależności od przypisanej roli. Użytkownik nieposiadający konta może przejść do formularza rejestracji klikając przycisk Zarejestruj się.

The image shows a screenshot of a web application window titled "LoginWindow". The window has a light gray background. In the center, there is a white rectangular box containing the login form. The form is titled "Logowanie" in bold black text. Below the title, there are two labels: "Email:" and "Hasło:". Each label is followed by a light gray rounded rectangular input field. Below the input fields, there is a red rounded rectangular button with the text "Zaloguj się" in white. Below the button, the word "lub" is centered in a small gray font. At the bottom of the form, there is a red text link "Zarejestruj się". The window has standard OS window controls (minimize, maximize, close) in the top right corner.

Rys. 5.1. Widok panelu logowania.

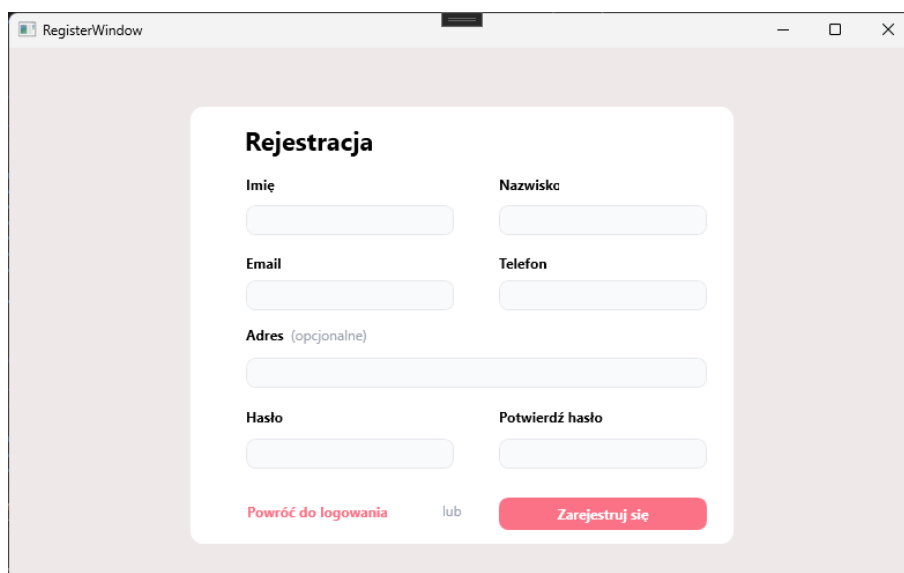
W przypadku gdy użytkownik nie wypełni pól formularza lub poda błędne dane logowania, wyświetlany jest komunikat błędu informujący o nieprawidłowym adresie e-mail lub hasle.



Rys. 5.2. Komunikat błędu przy nieprawidłowych danych logowania.

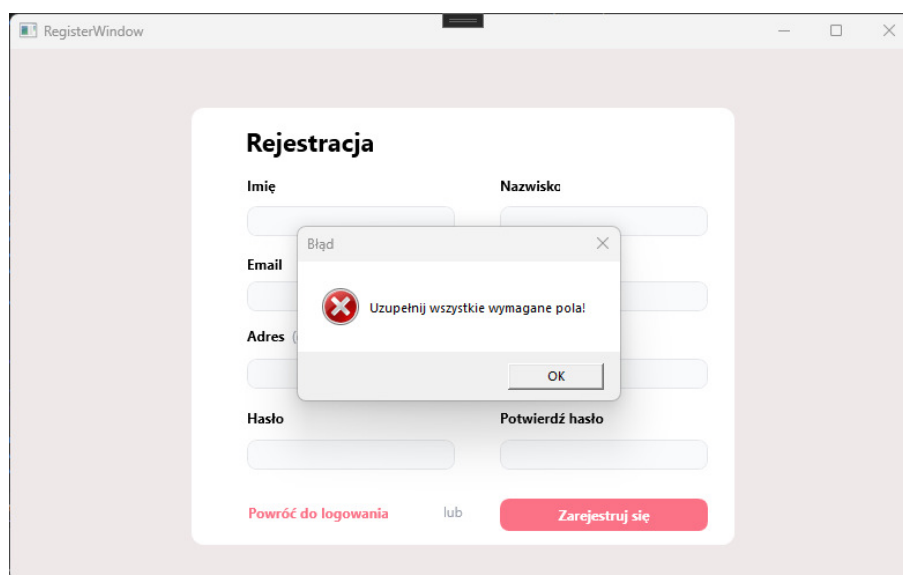
Panel rejestracji

Panel rejestracji umożliwia nowemu użytkownikowi założenie konta w systemie. Formularz zawiera pola: imię, nazwisko, adres e-mail, numer telefonu, adres (pole opcjonalne), hasło oraz potwierdzenie hasła. Po wypełnieniu formularza i kliknięciu przycisku Zarejestruj się aplikacja przeprowadza wieloetapową walidację danych. Użytkownik posiadający już konto może powrócić do ekranu logowania klikając przycisk Powrót do logowania.



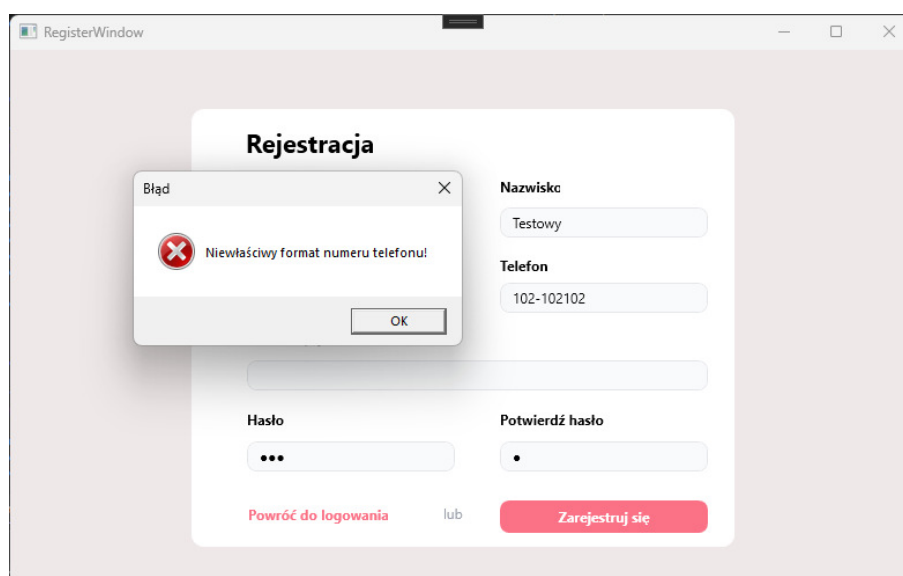
Rys. 5.3. Widok panelu rejestracji.

W przypadku gdy użytkownik nie wypełni wszystkich wymaganych pól formularza, wyświetlany jest komunikat błędu informujący o konieczności uzupełnienia danych.



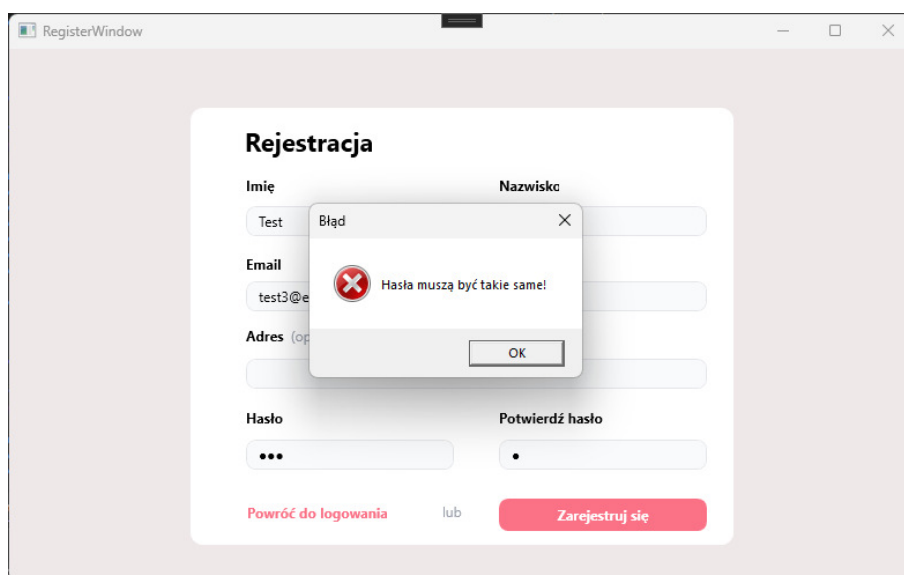
Rys. 5.4. Komunikat błędu przy niewypełnionych polach formularza.

Aplikacja weryfikuje również poprawność formatu numeru telefonu tzn. numer musi składać się z dokładnie 9 cyfr oraz nie zawierać innych znaków. W przypadku podania numeru w nieprawidłowym formacie wyświetlany jest odpowiedni komunikat błędu.



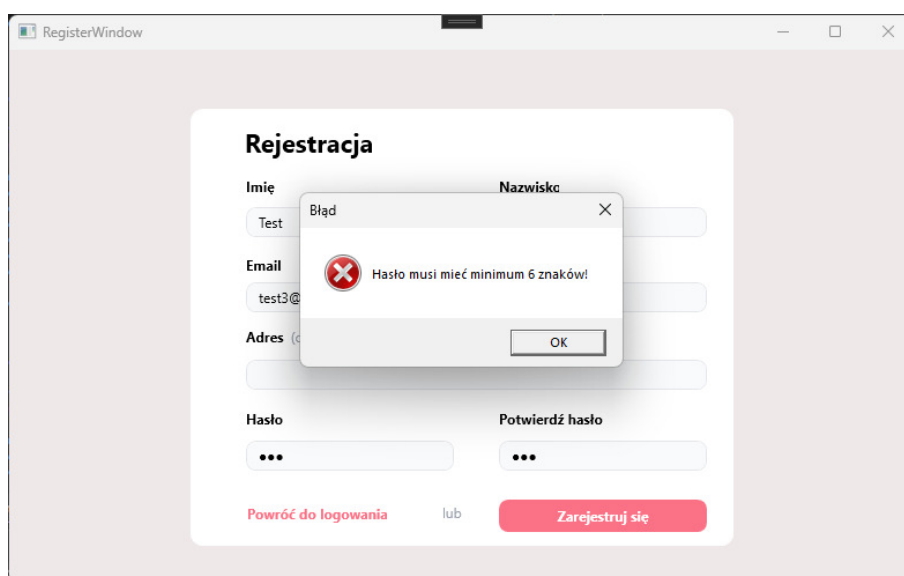
Rys. 5.5. Komunikat błędu przy nieprawidłowym formacie numeru telefonu.

Kolejnym etapem walidacji jest sprawdzenie, czy podane hasło i jego potwierdzenie są identyczne. W przypadku niezgodności haseł wyświetlany jest komunikat błędu.



Rys. 5.6. Komunikat błędu przy niezgodności haseł.

Ostatnim etapem walidacji jest sprawdzenie długości hasła - musi ono zawierać co najmniej 6 znaków. W przypadku podania zbyt krótkiego hasła wyświetlany jest odpowiedni komunikat błędu.



Rys. 5.7. Komunikat błędu przy hasle krótszym niż 6 znaków.

5.2. Panel użytkownika

Zakładka "Moje konto"

Zakładka "Moje konto" umożliwia zalogowanemu użytkownikowi przeglądanie i edycję swoich danych osobowych oraz zmianę hasła. W górnej części ekranu widoczny jest pasek nawigacyjny z nazwą kliniki, imieniem zalogowanego użytkownika, aktualnym saldem konta oraz przyciskiem wylogowania. Sekcja Twoje dane wyświetla imię, nazwisko, adres e-mail, numer telefonu oraz adres użytkownika wczytane automatycznie po zalogowaniu. Przycisk Zmień dane umożliwia zapisanie wprowadzonych zmian.

The screenshot shows a web application window titled 'MainWindow'. The header includes the logo 'Klinika Weterynaryjna', a greeting 'Witaj, Oskar!', a balance 'Saldo: 1437', and a 'Wyloguj' button. A navigation bar contains links: 'Moje konto' (active), 'Moje pupile', 'Moje wizyty', 'Dodaj pupila', 'Zarezerwuj wizytę', and 'Dodaj środki'. The main content area is divided into two sections. The first section, 'Twoje dane', contains input fields for 'Imię' (Oskar), 'Nazwisko' (Kowalski), 'Email' (jan@example.com), 'Telefon' (500-100-200), and 'Adres' (ul. Dołowa 1, Rzeszów), followed by a 'Zmień dane' button. The second section, 'Zmień hasło', contains input fields for 'Obecne hasło', 'Nowe hasło', and 'Potwierdź hasło', followed by a 'Zmień hasło' button.

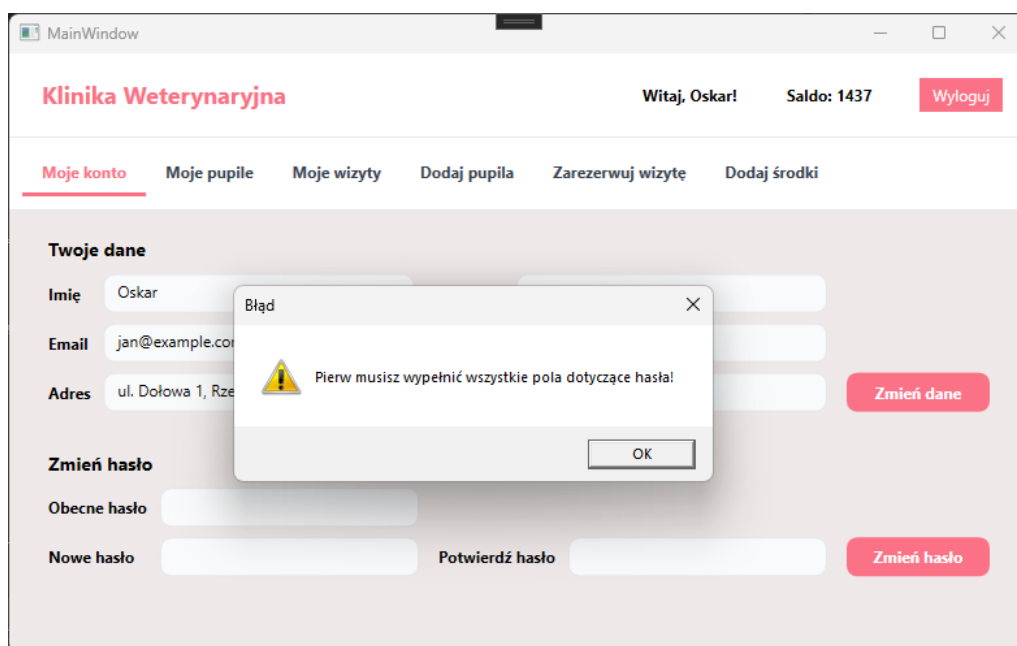
Rys. 5.8. Widok zakładki Moje konto.

W przypadku kliknięcia przycisku Zmień dane bez wprowadzenia żadnych modyfikacji, aplikacja wyświetla komunikat informujący o konieczności wprowadzenia danych do zmiany.

This screenshot shows the same 'Moje konto' page as in Rys. 5.8, but with an 'Info' dialog box overlaid in the center. The dialog box has a title bar with 'Info' and a close button. It contains an information icon and the text 'Wprowadź dane do zmiany!'. Below the text is an 'OK' button. The background page is dimmed, showing the same user details and password change sections.

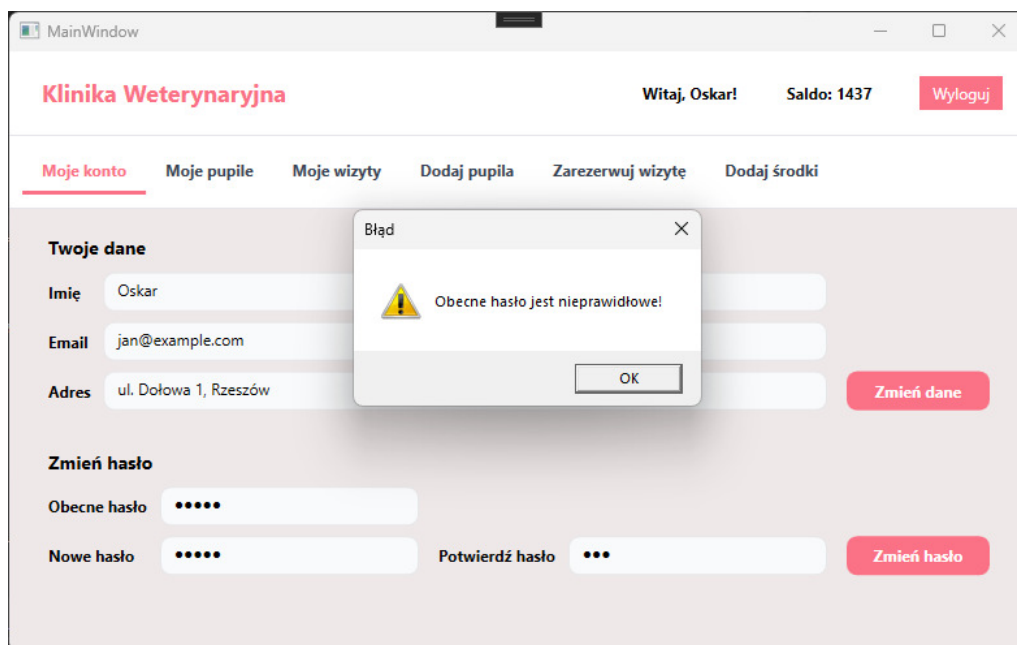
Rys. 5.9. Komunikat informacyjny przy braku zmian w danych.

Sekcja Zmień hasło zawiera trzy pola: obecne hasło, nowe hasło oraz potwierdzenie nowego hasła. W przypadku gdy użytkownik nie wypełni wszystkich pól i kliknie przycisk Zmień hasło, wyświetlany jest komunikat błędu.



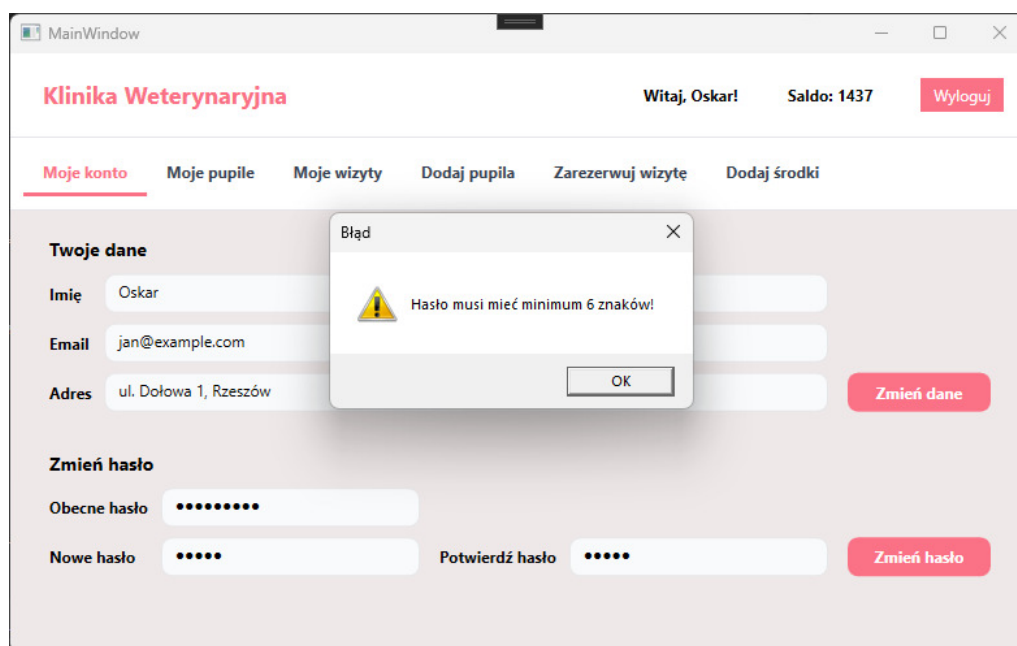
Rys. 5.10. Komunikat błędu przy niewypełnionych polach zmiany hasła.

Aplikacja weryfikuje również poprawność obecnego hasła jeśli podane hasło nie zgadza się z aktualnym hasłem zapisanym w systemie, wyświetlany jest komunikat błędu.

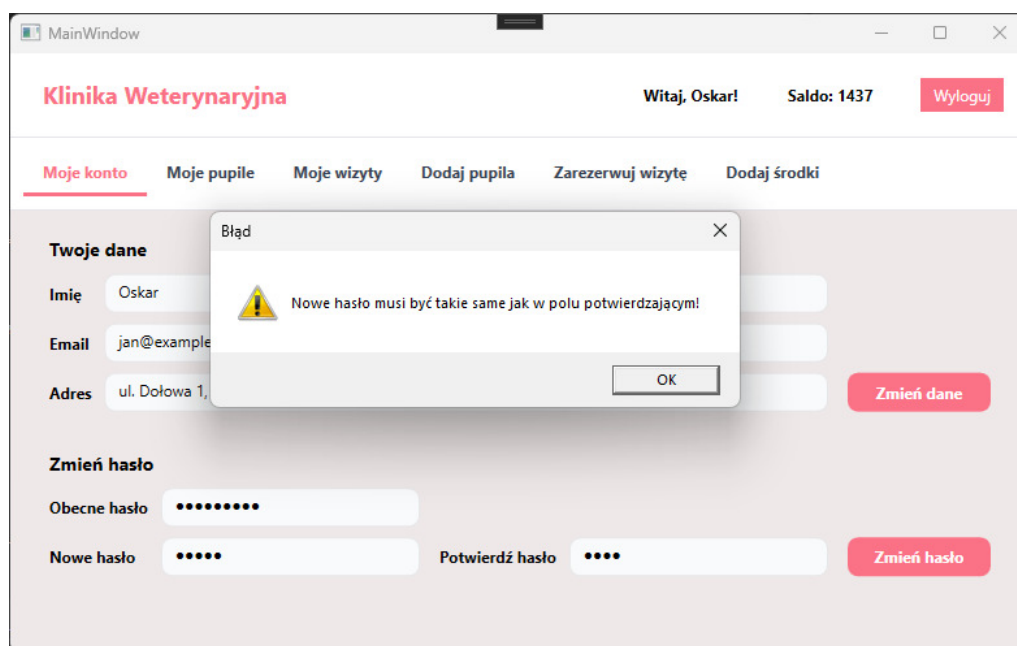


Rys. 5.11. Komunikat błędu przy nieprawidłowym obecnym hasle.

Kolejnym etapem walidacji jest sprawdzenie zgodności nowego hasła z polem potwierdzającym oraz sprawdzenie czy hasło ma co najmniej 6 znaków. W przypadku niezgodności lub zbyt krótkiego hasła wyświetlane są odpowiednie komunikaty błędów.



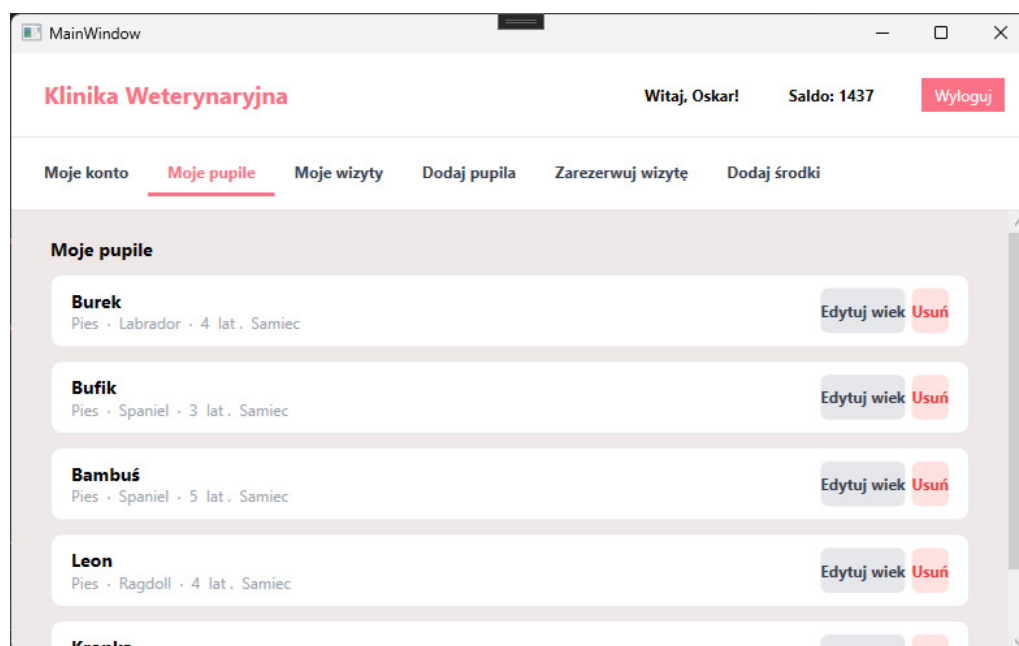
Rys. 5.12. Komunikat błędu przy niezgodności nowych haseł.



Rys. 5.13. Komunikat błędu przy hasle krótszym niż 6 znaków.

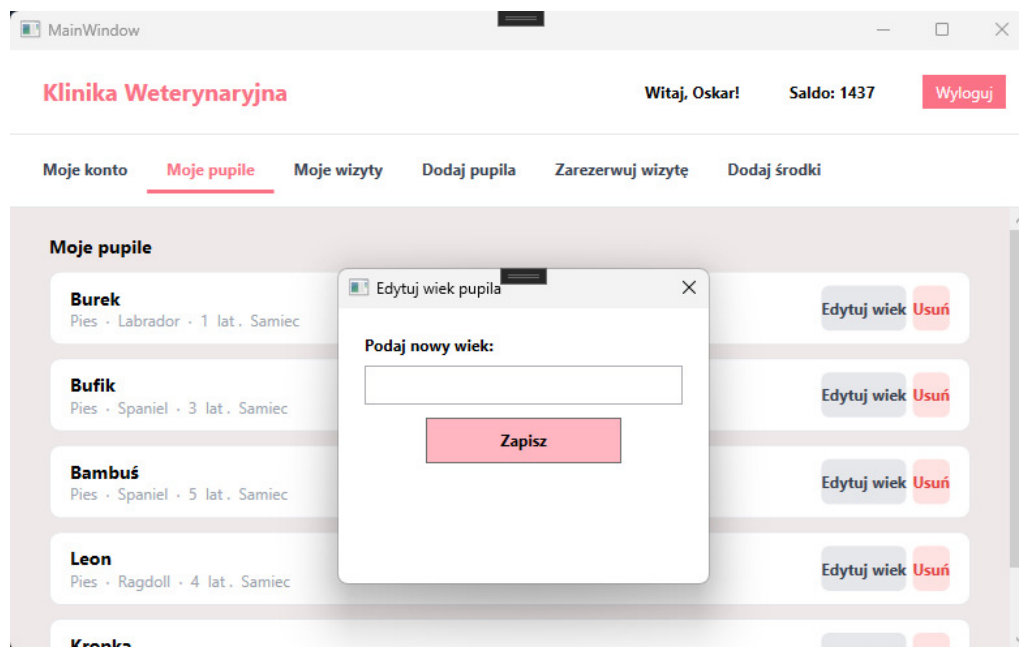
Zakładka "Moje pupile"

Zakładka "Moje pupile" wyświetla listę wszystkich pupili przypisanych do zalogowanego użytkownika. Dla każdego pupila prezentowane są informacje takie jak imię, gatunek, rasa, wiek oraz płeć. Przy każdym pupilu dostępne dwa przyciski: Edytuj wiek umożliwiający zmianę wieku pupila oraz Usuń umożliwiający usunięcie pupila z systemu.



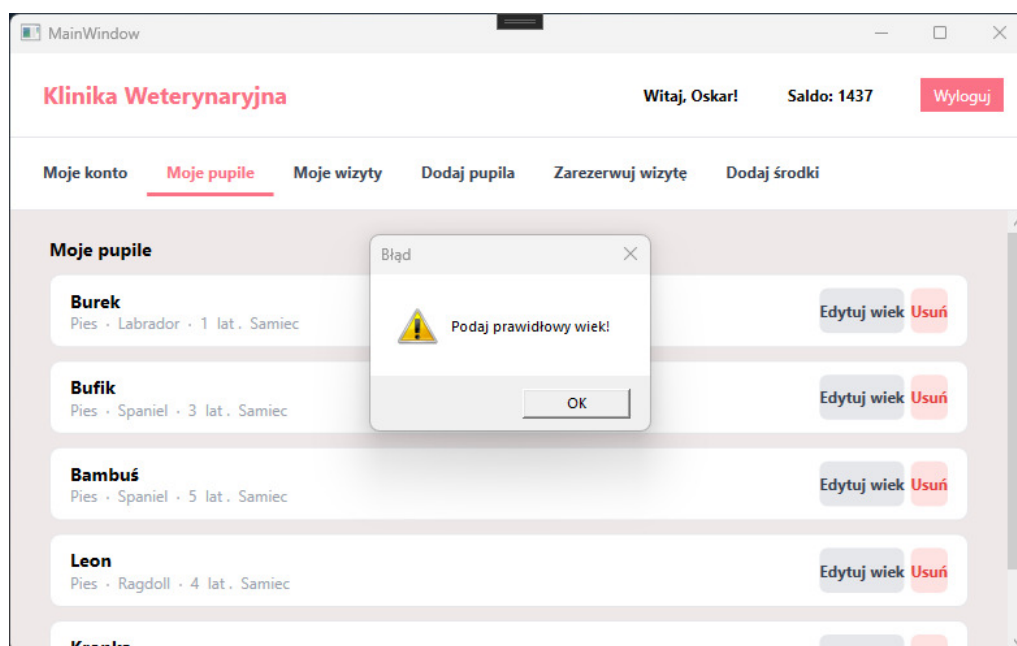
Rys. 5.14. Widok zakładki Moje pupile.

Po kliknięciu przycisku Edytuj wiek otwierane jest okno dialogowe z polem tekstowym, w którym użytkownik może wprowadzić nowy wiek pupila. Zmiana zostaje zapisana po kliknięciu przycisku Zapisz.



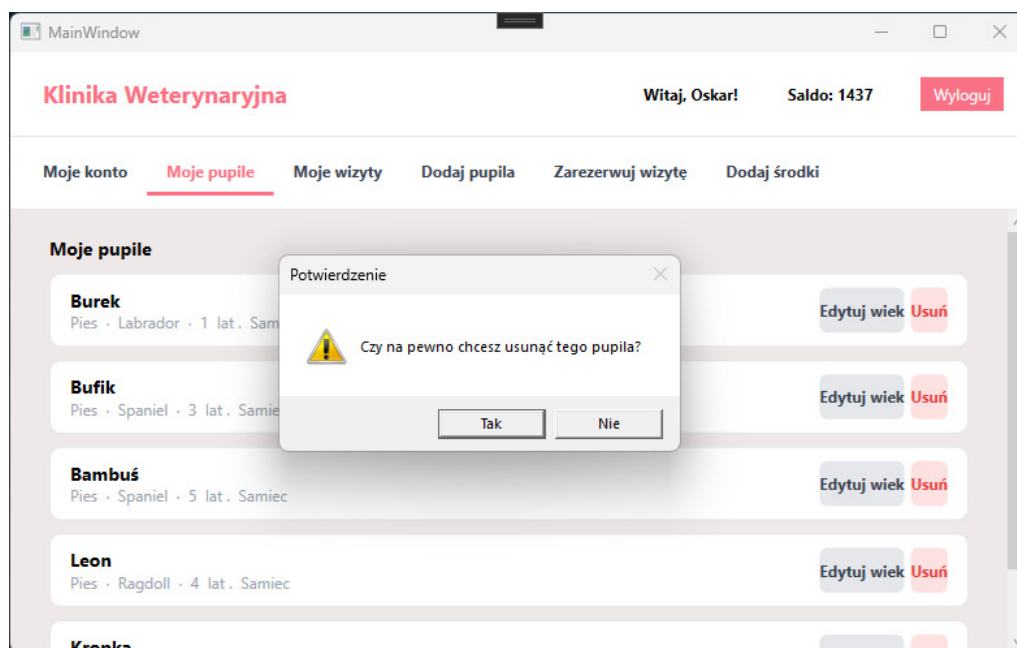
Rys. 5.15. Okno dialogowe edycji wieku pupila.

W przypadku podania nieprawidłowej wartości wieku aplikacja wyświetla komunikat błędu informujący o konieczności podania prawidłowej wartości.



Rys. 5.16. Komunikat błędu przy nieprawidłowym wieku pupila.

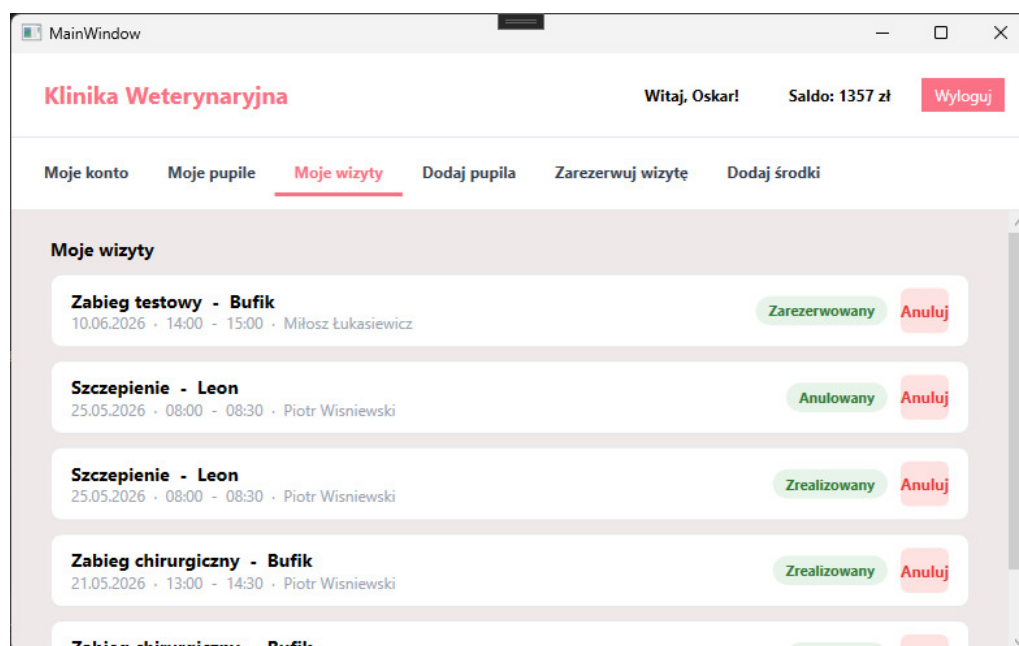
Po kliknięciu przycisku **Usuń** aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie operacji. Po potwierdzeniu pupil zostaje usunięty z systemu wraz z jego rezerwacjami, a lista pupilów jest automatycznie odświeżana.



Rys. 5.17. Okno potwierdzenia usunięcia pupila.

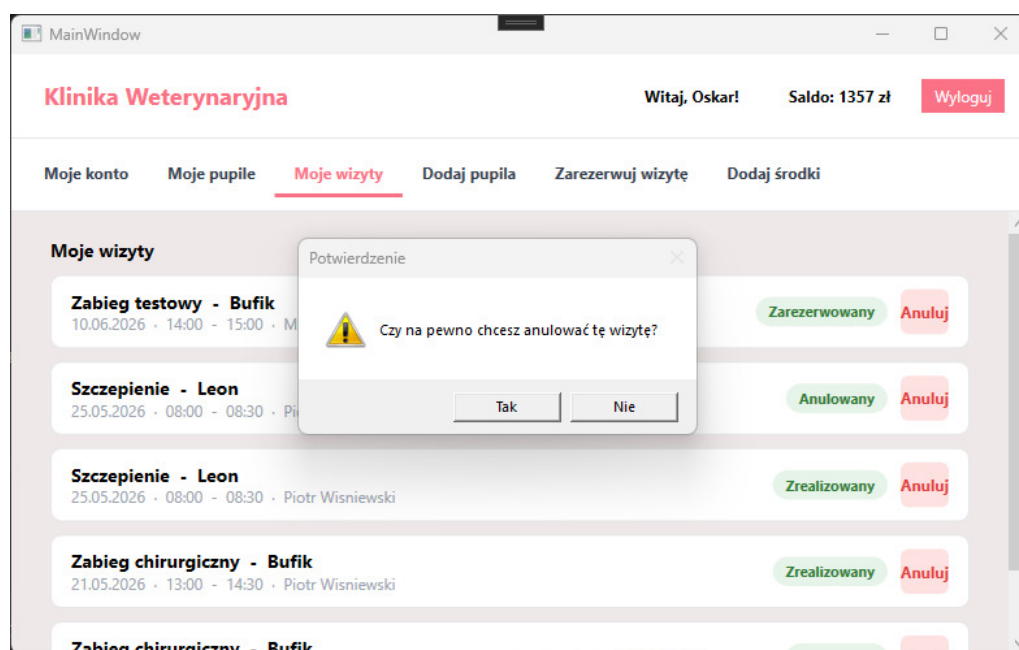
Zakładka "Moje wizyty"

Zakładka "Moje Wizyty" wyświetla historię wszystkich rezerwacji wizyt zalogowanego użytkownika. Dla każdej wizyty prezentowane są informacje takie jak nazwa zabiegu, imię pupila, data, godziny wizyty oraz imię i nazwisko weterynarza. Każda wizyta oznaczona jest kolorowym znacznikiem statusu: zielonym dla wizyt zarezerwowanych, szarym dla anulowanych oraz ciemnozielonym dla zrealizowanych. Przy każdej wizycie dostępny jest przycisk **Anuluj** umożliwiający anulowanie rezerwacji.



Rys. 5.18. Widok zakładki Moje wizyty.

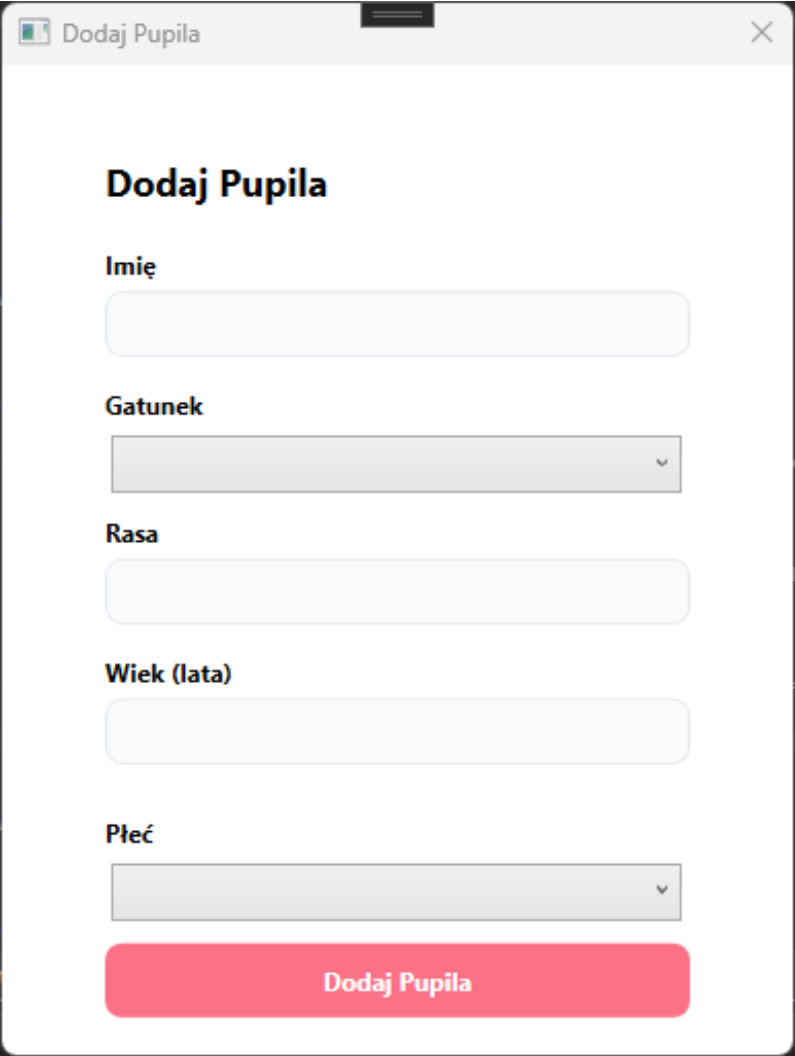
Po kliknięciu przycisku Anuluj aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie operacji. Po potwierdzeniu status wizyty zostaje zmieniony na Anulowany, a lista wizyt jest automatycznie odświeżana.



Rys. 5.19. Okno potwierdzenia anulowania wizyty.

Okno "Dodaj pupila"

Okno "Dodaj pupila" otwierane jest po kliknięciu przycisku Dodaj pupila w pasku nawigacyjnym. Formularz zawiera pola: imię, gatunek (lista rozwijana z opcjami Pies i Kot), rasa, wiek w latach oraz płeć (lista rozwijana z opcjami Samiec i Samica). Po wypełnieniu formularza i kliknięciu przycisku Dodaj Pupila aplikacja przeprowadza walidację wprowadzonych danych.



Dodaj Pupila

Imię

Gatunek

Rasa

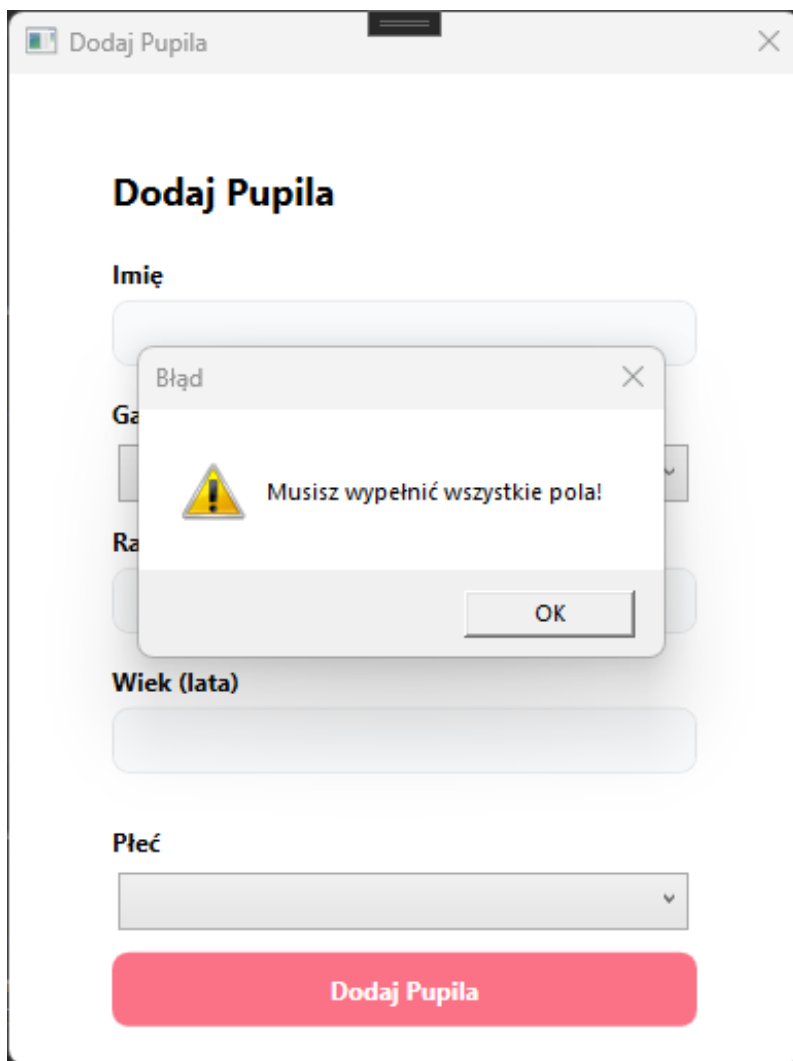
Wiek (lata)

Płeć

Dodaj Pupila

Rys. 5.20. Widok okna Dodaj pupila.

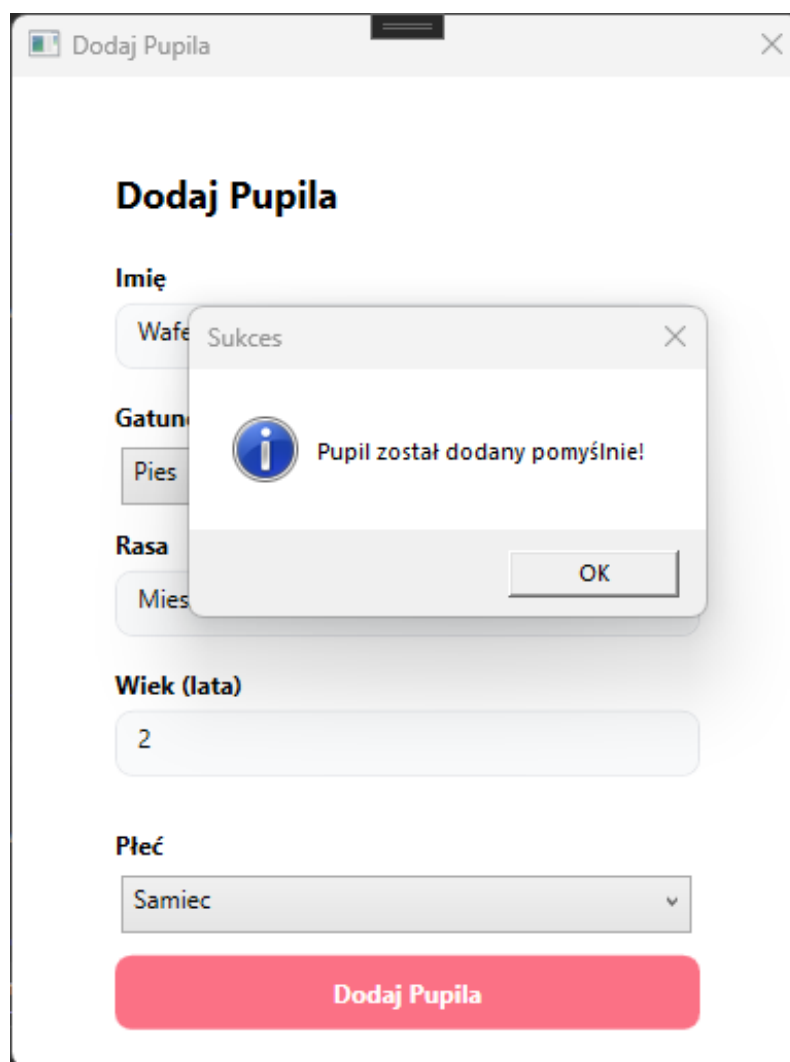
W przypadku gdy użytkownik nie wypełni wszystkich pól formularza, aplikacja wyświetla komunikat błędu informujący o konieczności uzupełnienia wszystkich pól.



The image shows a web application window titled "Dodaj Pupila". Inside the window, there is a form with the following fields: "Imię" (Name), "Gatunek" (Species), "Rasa" (Breed), "Wiek (lata)" (Age in years), and "Płeć" (Gender). Each field has a corresponding input box. At the bottom of the form is a red button labeled "Dodaj Pupila". Overlaid on the form is a small error dialog box titled "Błąd" (Error). The dialog box contains a yellow warning triangle icon and the text "Musisz wypełnić wszystkie pola!" (You must fill in all fields!). There is an "OK" button at the bottom right of the dialog box.

Rys. 5.21. Komunikat błędu przy niewypełnionych polach formularza.

Po poprawnym wypełnieniu wszystkich pól i kliknięciu przycisku Dodaj Pupila nowy pupil zostaje zapisany w bazie danych i przypisany do zalogowanego użytkownika. Aplikacja wyświetla komunikat potwierdzający pomyślne dodanie pupila, a okno zostaje zamknięte.



Rys. 5.22. Komunikat potwierdzający pomyślne dodanie pupila.

Okno "Nowa rezerwacja"

Okno "Nowa rezerwacja" otwierane jest po kliknięciu przycisku Zarezerwuj wizytę w pasku nawigacyjnym. Proces rezerwacji podzielony jest na trzy kroki. W pierwszym kroku użytkownik wybiera pupila z listy rozwijanej oraz zabieg z listy dostępnych zabiegów. Dla każdego zabiegu wyświetlany jest czas trwania oraz cena.

The screenshot shows a web application window titled 'Nowa rezerwacja'. Below the title is the subtitle 'Krok 1 z 3 - Wybierz pupila i zabieg'. The main section is titled 'Wybierz pupila' and contains a dropdown menu. Below this is another section titled 'Wybierz zabieg' which lists four procedures with their durations and prices:

Zabieg	Czas trwania	Cena
Szczepienie	30 min	80.00 zł
Kontrola ogólna	45 min	120.00 zł
Zabieg chirurgiczny	90 min	350.00 zł
Czyszczenie zębów	60 min	200.00 zł

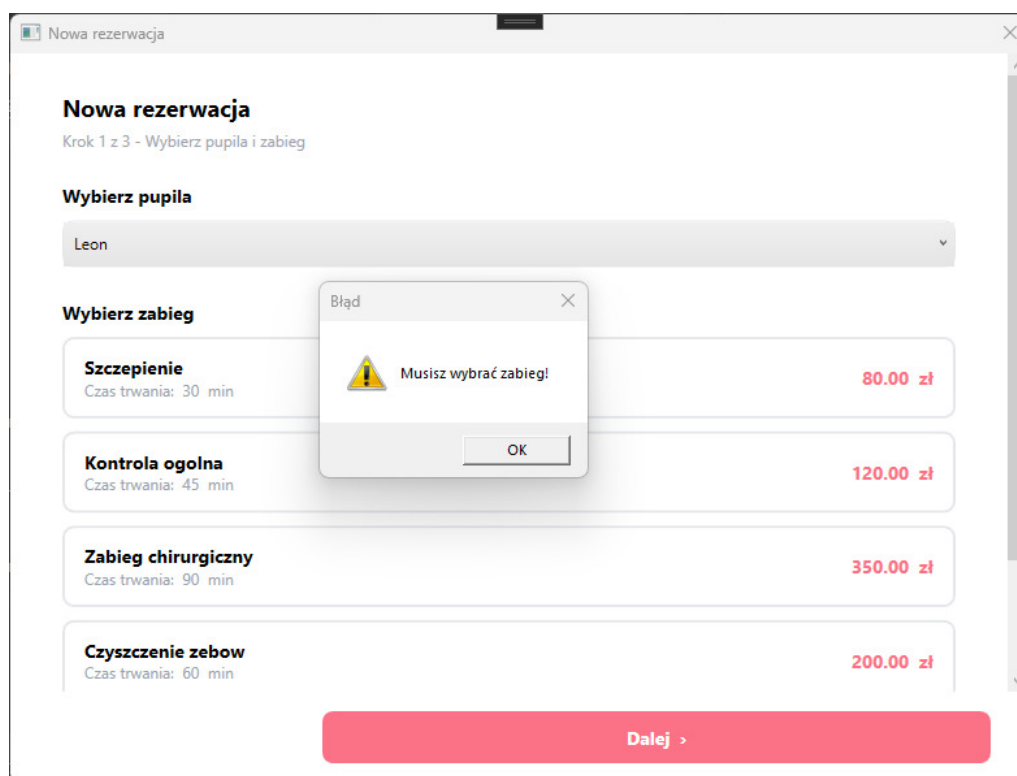
At the bottom of the form is a red button labeled 'Dalej >'.

Rys. 5.23. Krok 1 - widok wyboru pupila i zabiegu.

W przypadku gdy użytkownik nie wybierze pupila przed kliknięciem przycisku Dalej, aplikacja wyświetla komunikat błędu. Analogicznie, jeśli użytkownik nie wybierze zabiegu, wyświetlany jest odpowiedni komunikat.

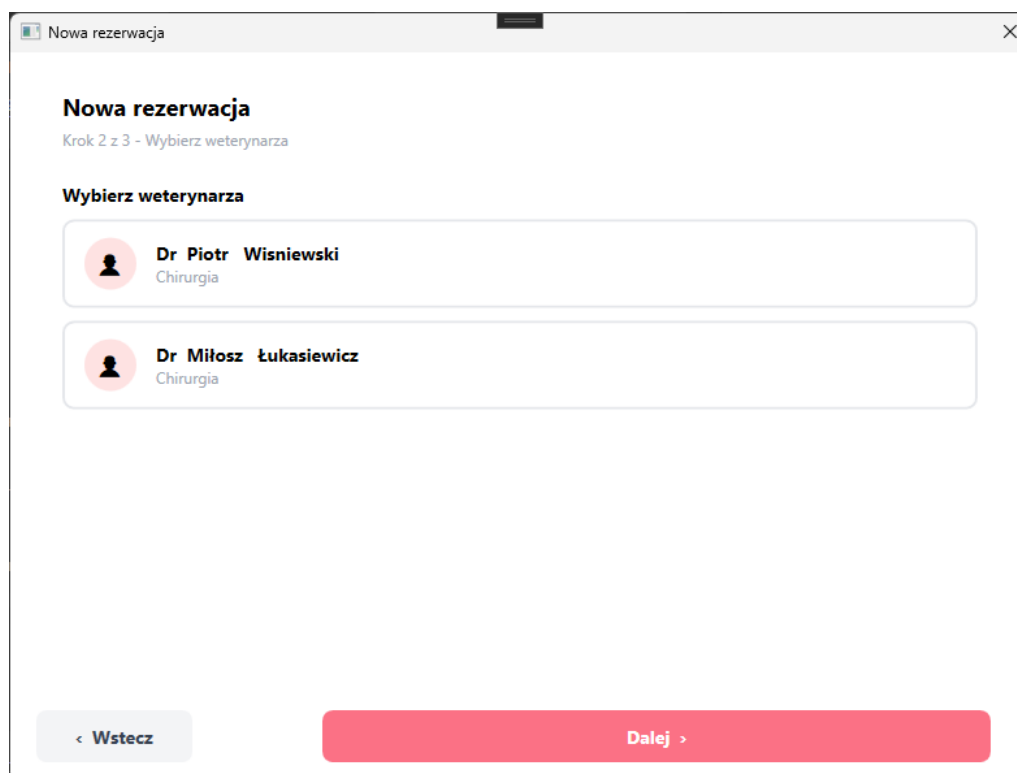
This screenshot shows the same 'Nowa rezerwacja' form as in Rys. 5.23, but with an error dialog box overlaid. The dialog box is titled 'Błąd' and contains a yellow warning icon and the text 'Musisz wybrać pupila!'. There is an 'OK' button at the bottom of the dialog. The background form is slightly dimmed.

Rys. 5.24. Komunikat błędu przy braku wybranego pupila.



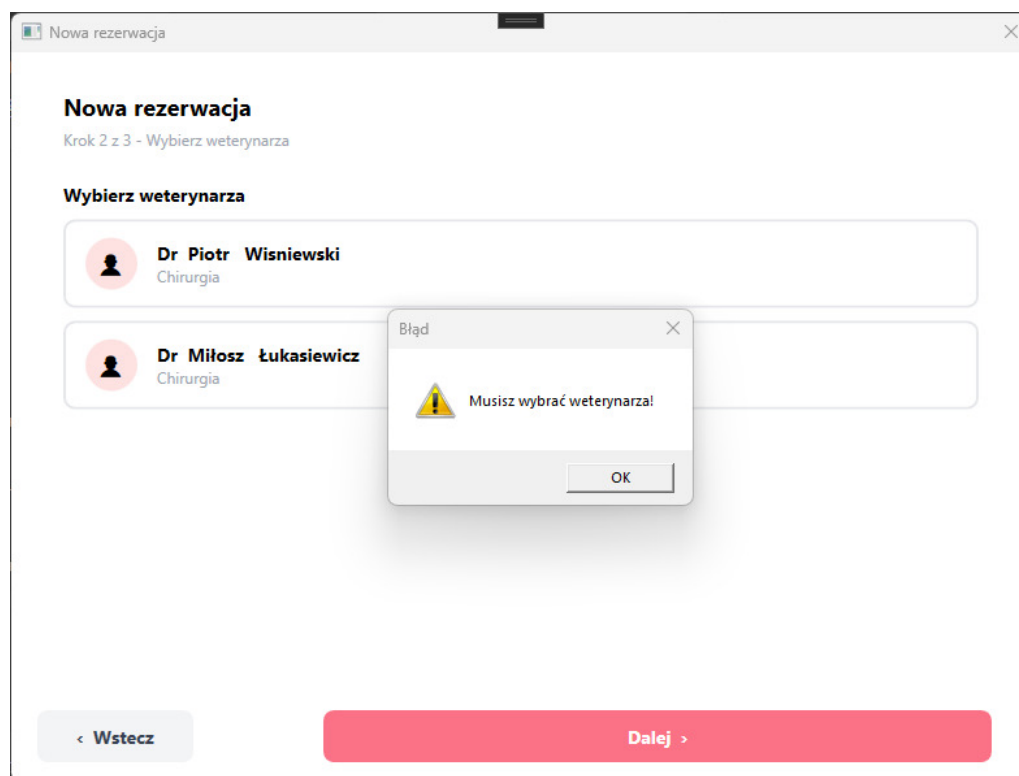
Rys. 5.25. Komunikat błędu przy braku wybranego zabiegu.

W drugim kroku użytkownik wybiera weterynarza spośród tych, którzy wykonują wybrany zabieg. Dla każdego weterynarza wyświetlane jest imię, nazwisko oraz specjalizacja.



Rys. 5.26. Krok 2 - widok wyboru weterynarza.

W przypadku gdy użytkownik nie wybierze weterynarza przed kliknięciem przycisku Dalej, aplikacja wyświetla komunikat błędu.



Rys. 5.27. Komunikat błędu przy braku wybranego weterynarza.

W trzecim kroku użytkownik wybiera termin wizyty z kalendarza dostępności wybranego weterynarza. Kalendarz prezentuje tygodniowy widok z podziałem na godziny. Sloty oznaczone kolorem zielonym są wolne i możliwe do zarezerwowania, czerwonym zajęte, natomiast szare oznaczają terminy poza godzinami pracy weterynarza lub terminy w których zabieg nie zmieści się przed końcem pracy.

Nowa rezerwacja
Krok 3 z 3 - Wybierz termin

Wybierz termin

	Pt 5.06	Wt 9.06	Śr 10.06
8:00	Wolne	Wolne	Wolne
9:00	Wolne	Wolne	Wolne
10:00	Wolne	Wolne	Wolne
11:00	Wolne	Wolne	Wolne
12:00	Wolne	Wolne	Wolne
13:00	Wolne	Wolne	Wolne
14:00	-	-	-
15:00	-	-	-

< Wstecz Zarezerwuj

Rys. 5.28. Krok 3 - widok kalendarza dostępności weterynarza.

W przypadku gdy użytkownik nie wybierze terminu przed kliknięciem przycisku Zarezerwuj, aplikacja wyświetla komunikat błędu.

Nowa rezerwacja
Krok 3 z 3 - Wybierz termin

Wybierz termin

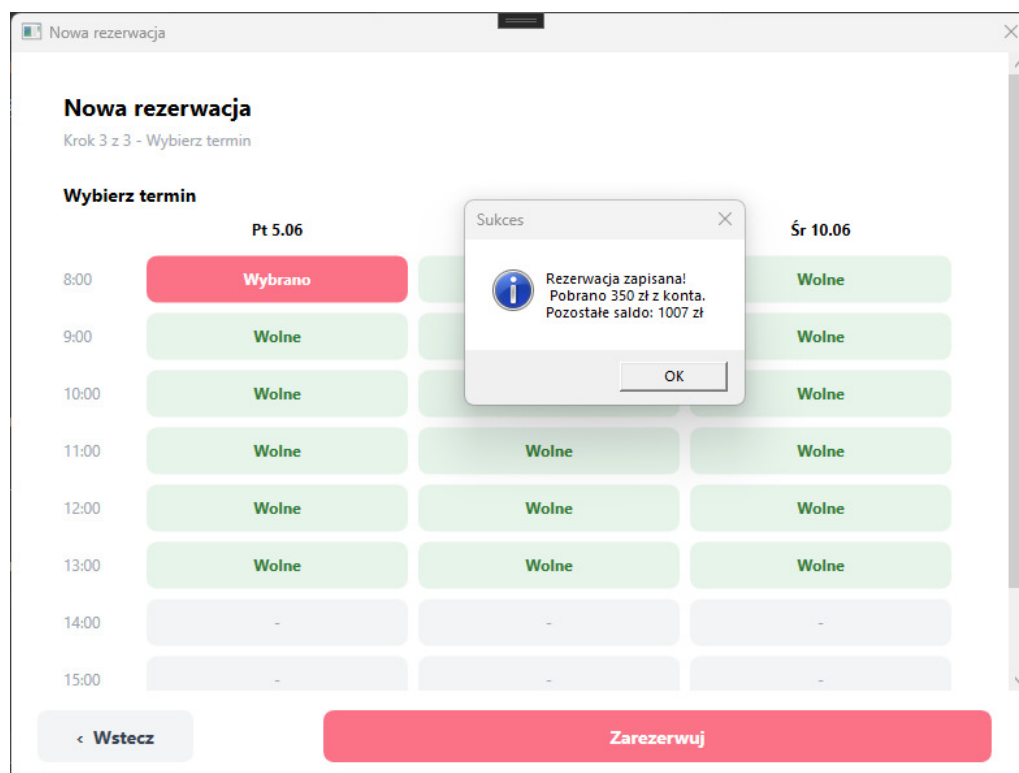
	Pt 5.06	Wt 9.06	Śr 10.06
8:00	Zajęte	Wolne	Wolne
9:00	Zajęte	Zajęte	Wolne
10:00	Wolne	Wolne	Wolne
11:00	Wolne	Wolne	Wolne
12:00	Wolne	Wolne	Wolne
13:00	Wolne	Wolne	Wolne
14:00	Wolne	Wolne	Wolne
15:00	-	-	-

Błąd
Musisz wybrać termin!
OK

< Wstecz Zarezerwuj

Rys. 5.29. Komunikat błędu przy braku wybranego terminu.

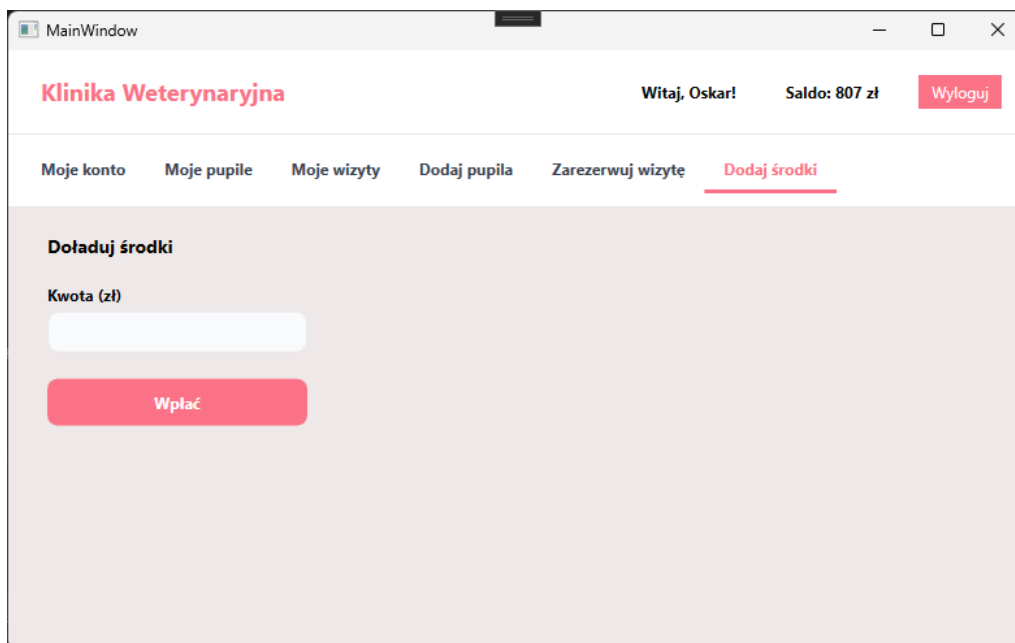
Po wybraniu wolnego terminu i kliknięciu przycisku Zarezerwuj rezerwacja zostaje zapisana w bazie danych, a z konta użytkownika pobierana jest kwota odpowiadająca cenie zabiegu. Aplikacja wyświetla komunikat potwierdzający pomyślne zapisanie rezerwacji wraz z informacją o pobranej kwocie oraz pozostałym saldzie.



Rys. 5.30. Komunikat potwierdzający pomyślne zapisanie rezerwacji.

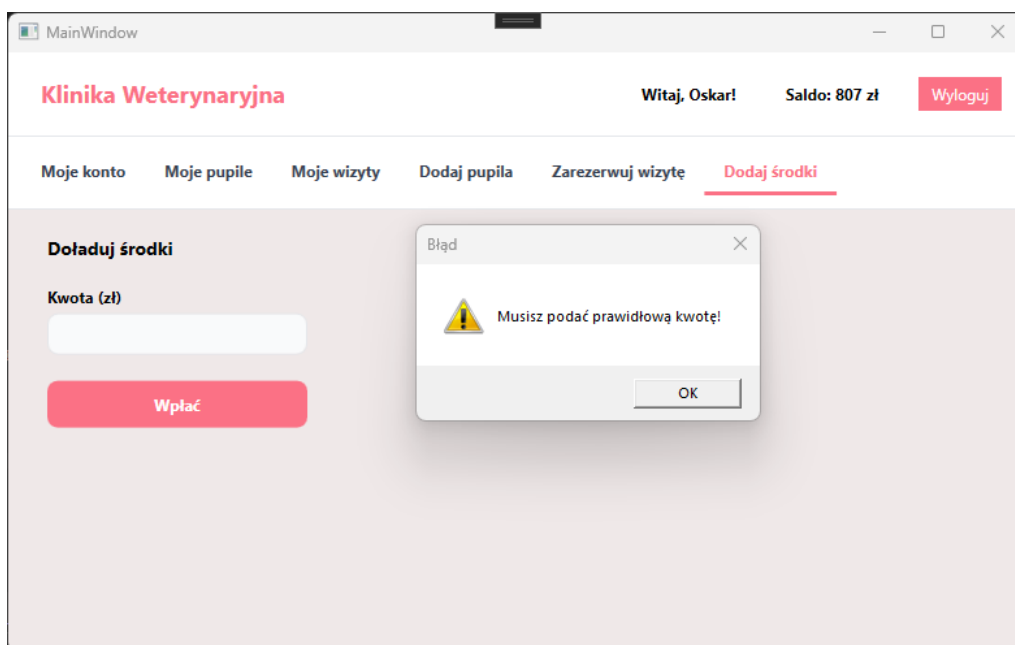
Zakładka "Dodaj środki"

Zakładka "Dodaj środki" umożliwia użytkownikowi doładowanie salda swojego konta. Formularz zawiera jedno pole tekstowe do wprowadzenia kwoty wpłaty oraz przycisk Wpłać. Aktualne saldo konta wyświetlane jest na bieżąco w górnym pasku aplikacji.



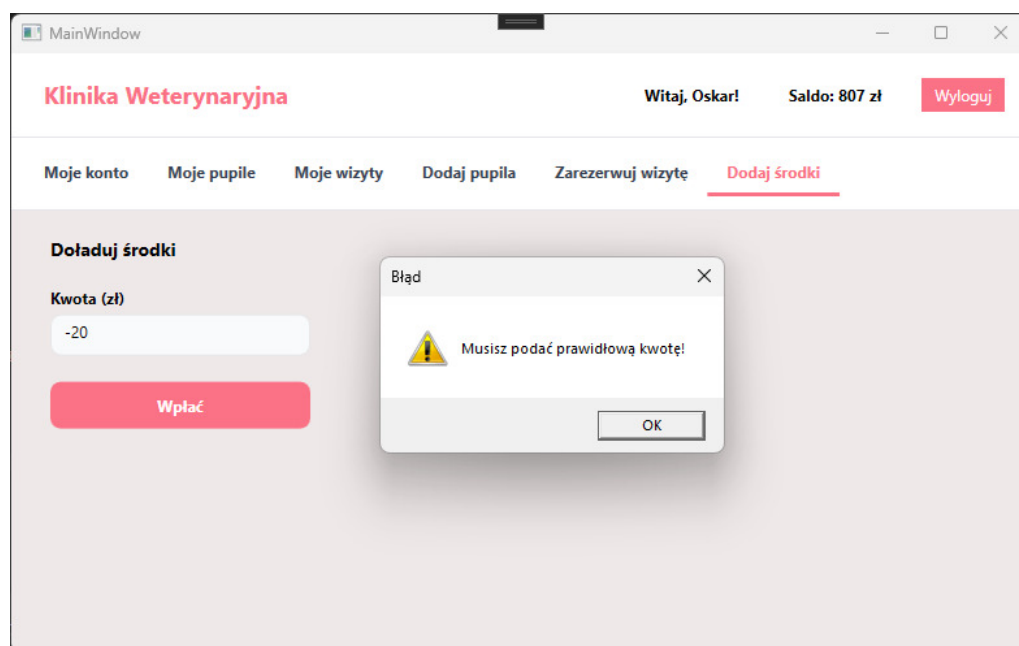
Rys. 5.31. Widok zakładki Dodaj środki.

W przypadku gdy użytkownik nie wprowadzi żadnej kwoty i kliknie przycisk Wpłać, aplikacja wyświetla komunikat błędu informujący o konieczności podania prawidłowej kwoty.



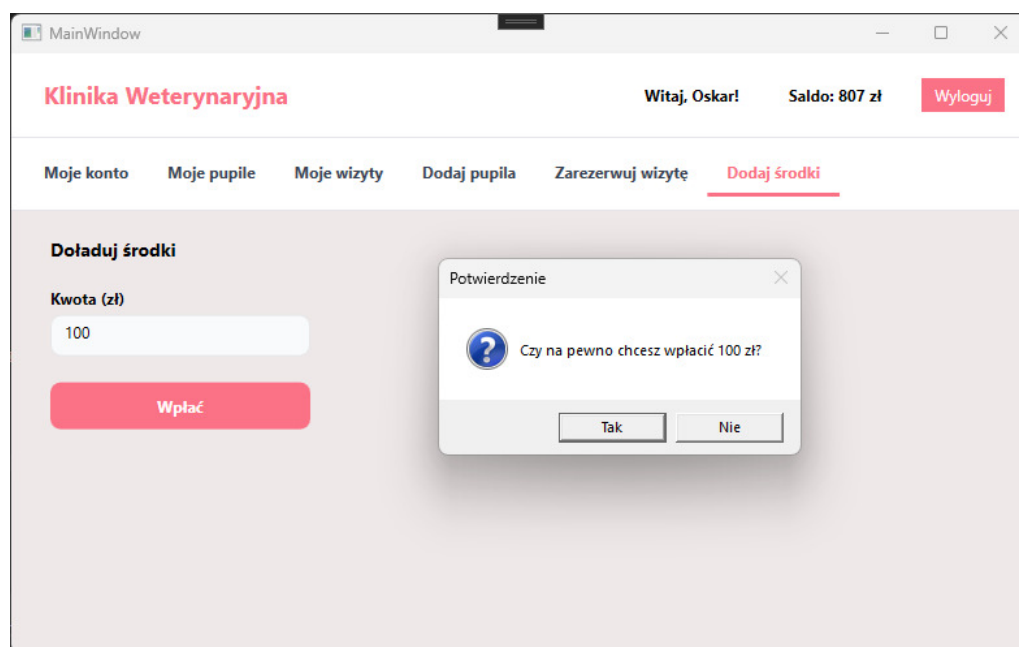
Rys. 5.32. Komunikat błędu przy pustym polu kwoty.

Aplikacja waliduje również podaną kwotę - nie może ona być liczbą ujemną ani równą zero. W przypadku podania nieprawidłowej wartości wyświetlany jest komunikat błędu.



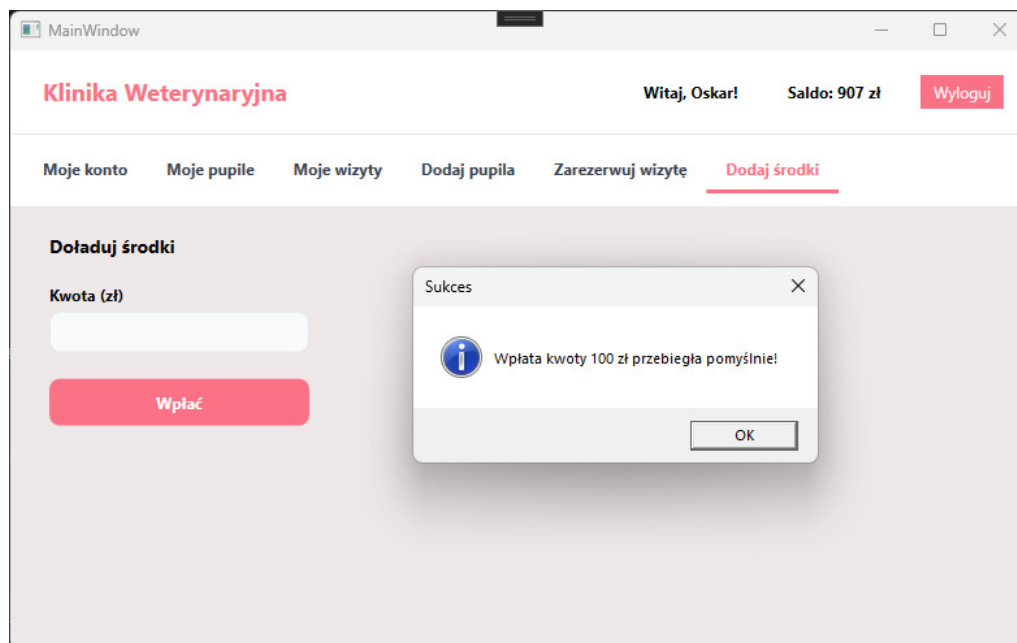
Rys. 5.33. Komunikat błędu przy podaniu nieprawidłowej kwoty.

Po wprowadzeniu prawidłowej kwoty i kliknięciu przycisku Wpłać aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie operacji, informując o kwocie która zostanie dodana do salda.



Rys. 5.34. Okno potwierdzenia wpłaty środków.

Po zatwierdzeniu operacji saldo konta użytkownika zostaje zwiększone o podaną kwotę, zarówno w bazie danych jak i w interfejsie aplikacji. Aplikacja wyświetla komunikat potwierdzający pomyślne wykonanie wpłaty, a pole tekstowe zostaje wyczyszczone.

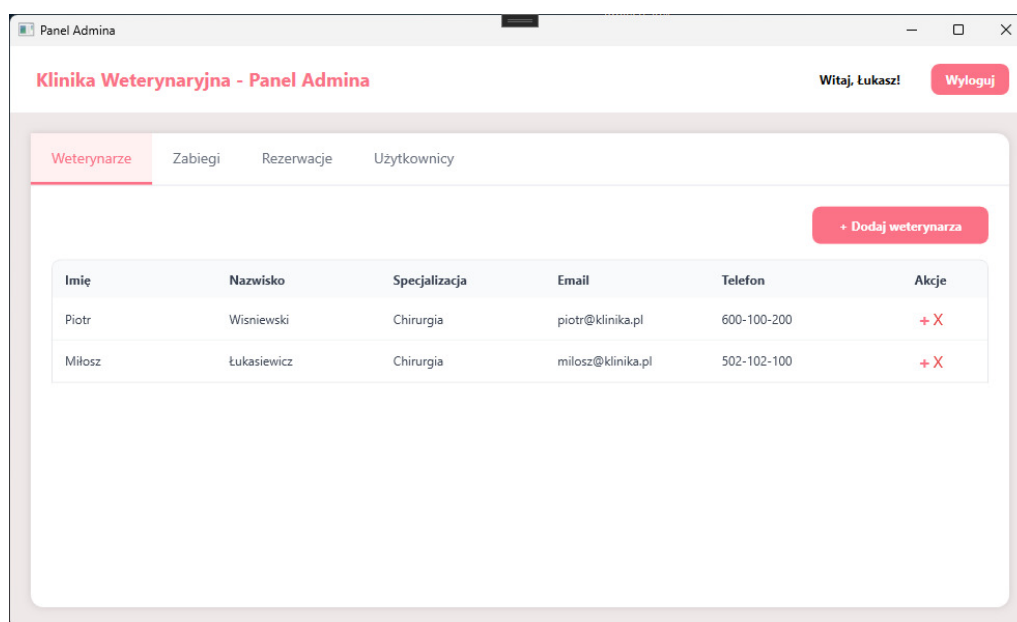


Rys. 5.35. Komunikat potwierdzający pomyślną wpłatę środków.

5.3. Panel administratora

Zakładka "Weterynarze"

Zakładka "Weterynarze" wyświetla listę wszystkich weterynarzy zarejestrowanych w systemie. Dla każdego weterynarza prezentowane są informacje takie jak imię, nazwisko, specjalizacja, adres e-mail oraz numer telefonu. Przy każdym weterynarzu dostępne są dwa przyciski akcji: + umożliwiający zarządzanie zabiegami przypisanymi do weterynarza oraz X umożliwiający usunięcie weterynarza z systemu. W prawym górnym rogu zakładki znajduje się przycisk (+ Dodaj weterynarza).



Rys. 5.36. Widok zakładki Weterynarze w panelu administratora.

Po kliknięciu przycisku (+ Dodaj weterynarza) otwierane jest okno formularza. Formularz zawiera pola: imię, nazwisko, specjalizacja, email, telefon, hasło oraz sekcję Godziny pracy z polami Od i Do dla każdego dnia tygodnia od poniedziałku do piątku. Domyślne godziny pracy ustawione są na 08:00-15:00. Pozostawienie pól pustych dla danego dnia oznacza, że weterynarz nie pracuje w tym dniu.

Dodaj weterynarza

Imię

Nazwisko

Specjalizacja

Email

Telefon

Hasło

Godziny pracy

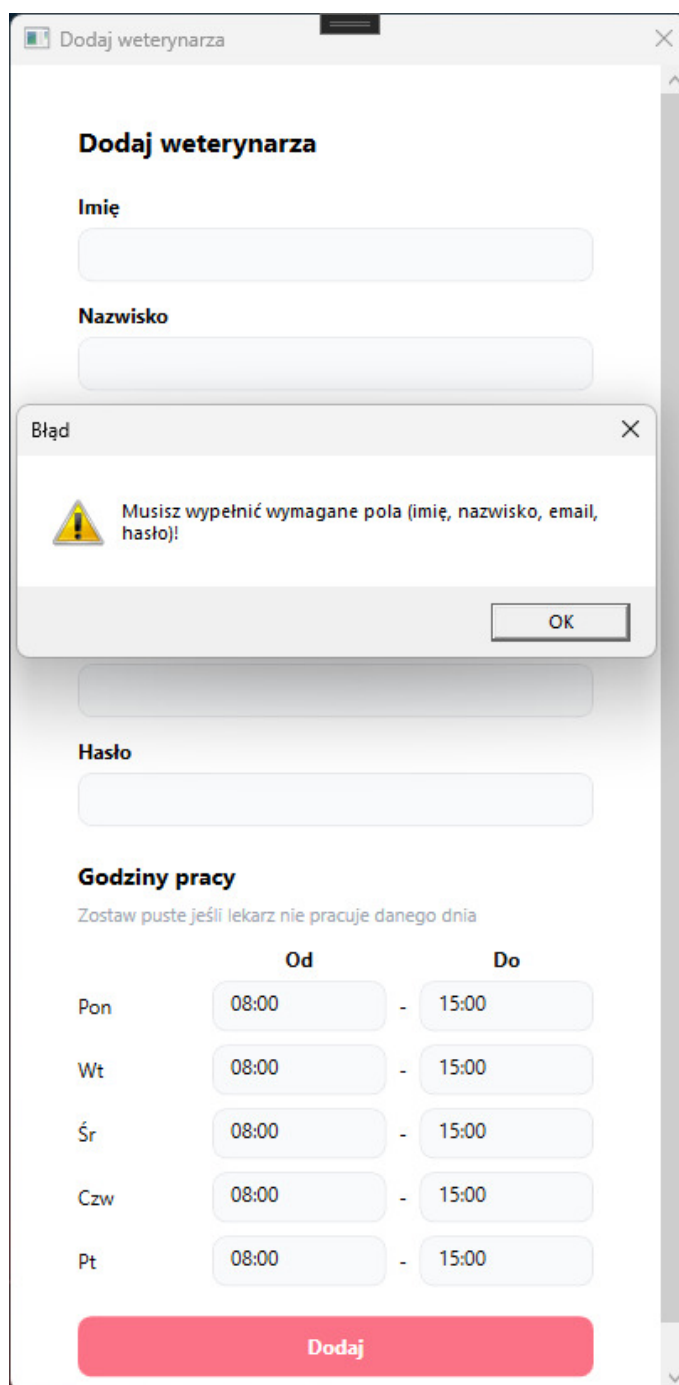
Zostaw puste jeśli lekarz nie pracuje danego dnia

	Od		Do
Pon	08:00	-	15:00
Wt	08:00	-	15:00
Śr	08:00	-	15:00
Czw	08:00	-	15:00
Pt	08:00	-	15:00

Dodaj

Rys. 5.37. Okno formularza dodawania weterynarza.

W przypadku gdy administrator nie wypełni wymaganych pól (imię, nazwisko, email, hasło) aplikacja wyświetla komunikat błędu.

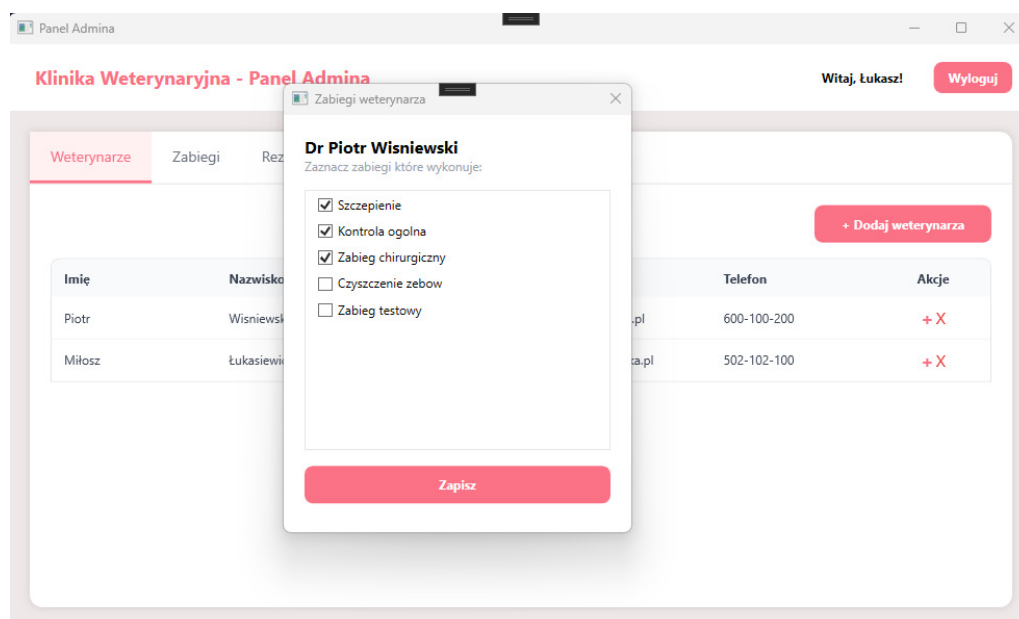


The screenshot shows a web application window titled "Dodaj weterynarza". The form contains several input fields: "Imię", "Nazwisko", "Email", "Hasło", and a section for "Godziny pracy" (Working hours) with a table for days of the week (Pon, Wt, Śr, Czw, Pt) and time slots (Od, Do). A red "Dodaj" button is at the bottom. An error dialog box is overlaid on the form, displaying a warning icon and the message: "Musisz wypełnić wymagane pola (imię, nazwisko, email, hasło)!" (You must fill in the required fields (name, surname, email, password)!). The dialog has an "OK" button.

	Od		Do
Pon	08:00	-	15:00
Wt	08:00	-	15:00
Śr	08:00	-	15:00
Czw	08:00	-	15:00
Pt	08:00	-	15:00

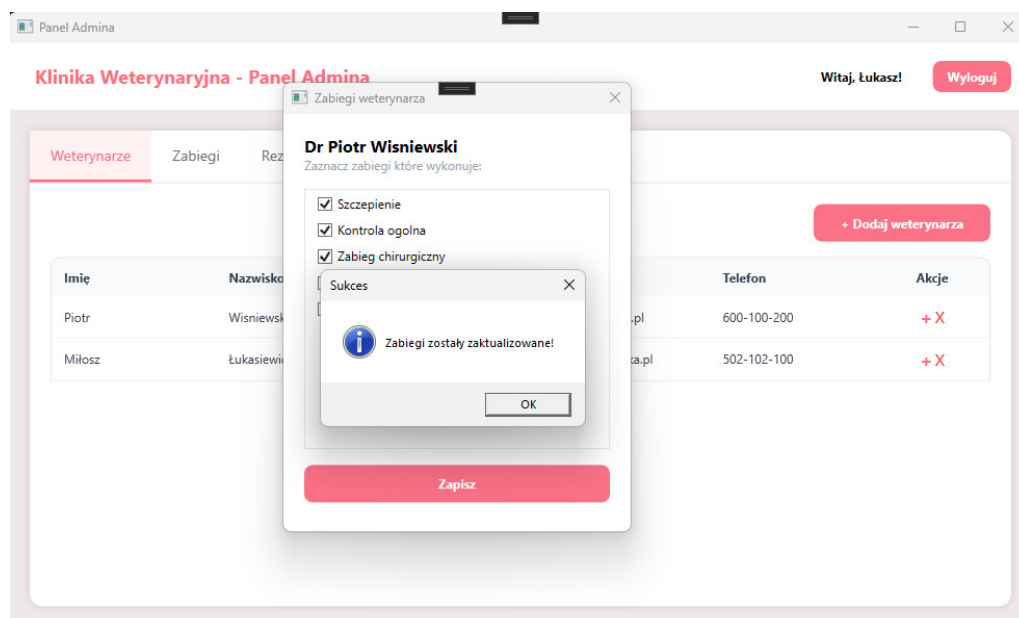
Rys. 5.38. Komunikat błędu przy niewypełnionych wymaganych polach.

Po kliknięciu przycisku + przy wybranym weterynarzu otwierane jest okno zarządzania zabiegami. Wyświetlana jest lista wszystkich dostępnych zabiegów z checkboxami, zaznaczone zabiegi oznaczają te, które dany weterynarz aktualnie wykonuje. Administrator może dowolnie zaznaczać i odznaczać zabiegi, a następnie zapisać zmiany przyciskiem Zapisz.



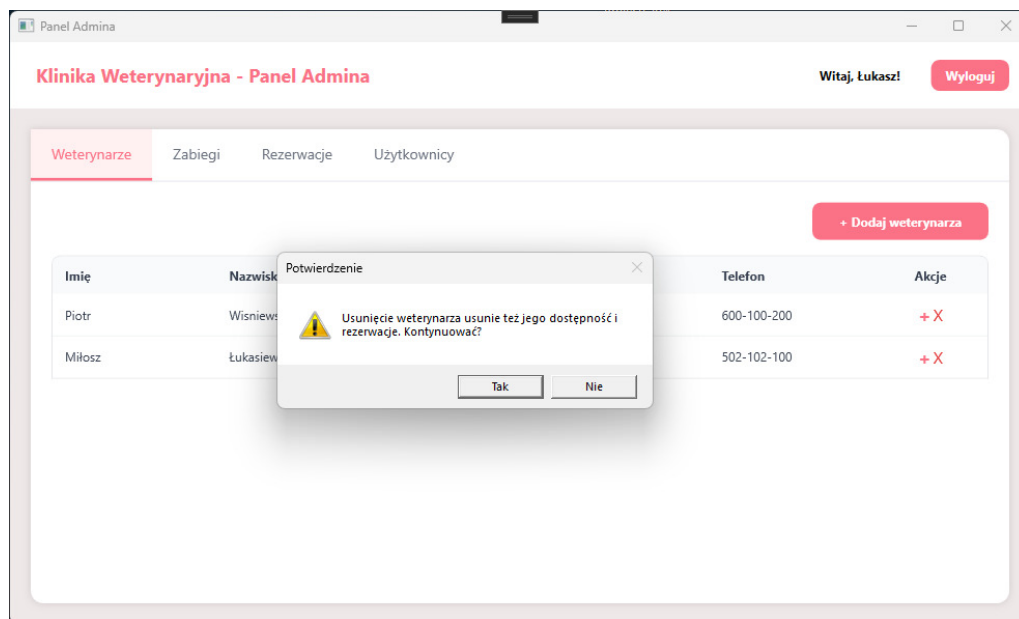
Rys. 5.39. Okno zarządzania zabiegami weterynarza.

Po kliknięciu przycisku Zapisz aplikacja aktualizuje przypisanie zabiegów do weterynarza w bazie danych i wyświetla komunikat potwierdzający pomyślne wykonanie operacji.



Rys. 5.40. Komunikat potwierdzający aktualizację zabiegów weterynarza.

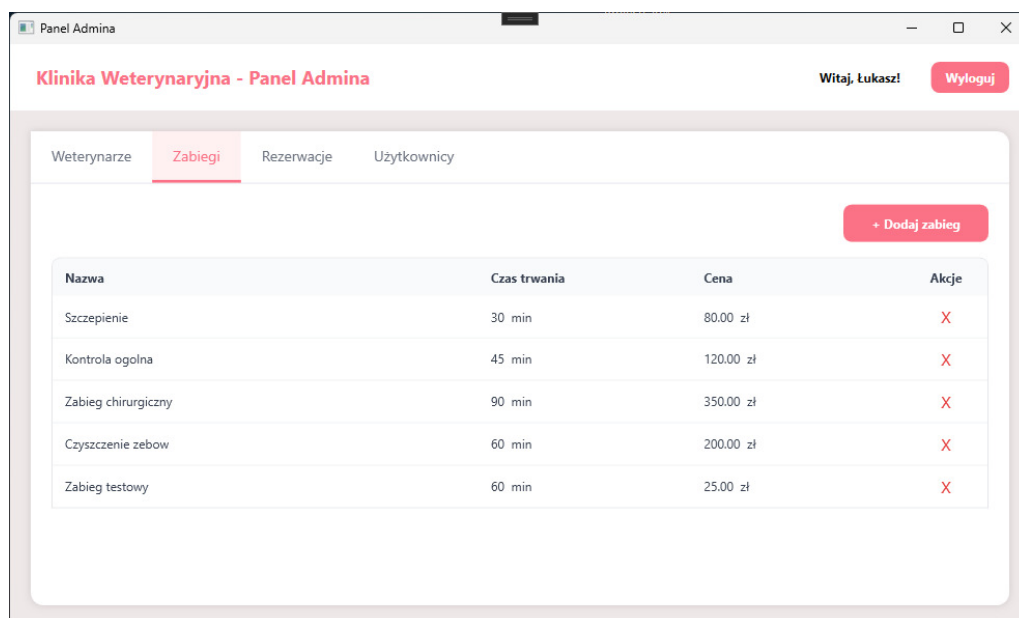
Po kliknięciu przycisku X przy wybranym weterynarzu aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie operacji, informując że usunięcie weterynarza spowoduje również usunięcie jego dostępności i rezerwacji. Po potwierdzeniu weterynarz zostaje trwale usunięty z systemu.



Rys. 5.41. Okno potwierdzenia usunięcia weterynarza.

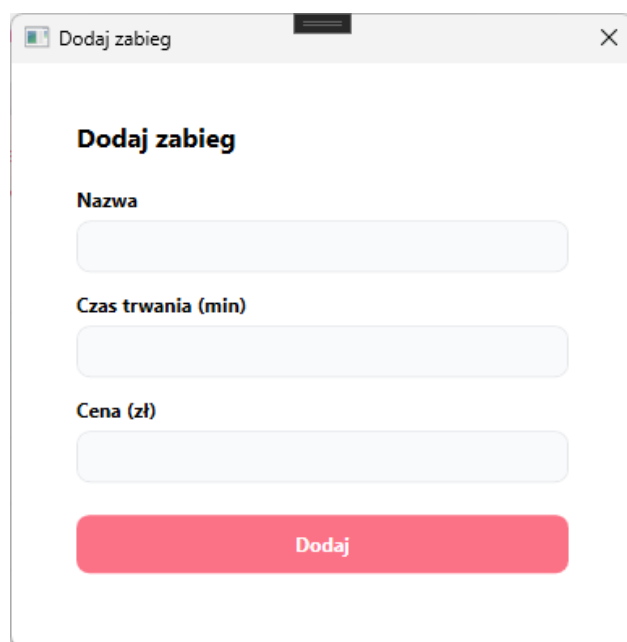
Zakładka "Zabiegi"

Zakładka "Zabiegi" wyświetla listę wszystkich zabiegów dostępnych w klinice. Dla każdego zabiegu prezentowane są informacje takie jak nazwa, czas trwania w minutach oraz cena. Przy każdym zabiegu dostępny jest przycisk X umożliwiający usunięcie zabiegu z systemu. W prawym górnym rogu zakładki znajduje się przycisk (+ Dodaj zabieg).



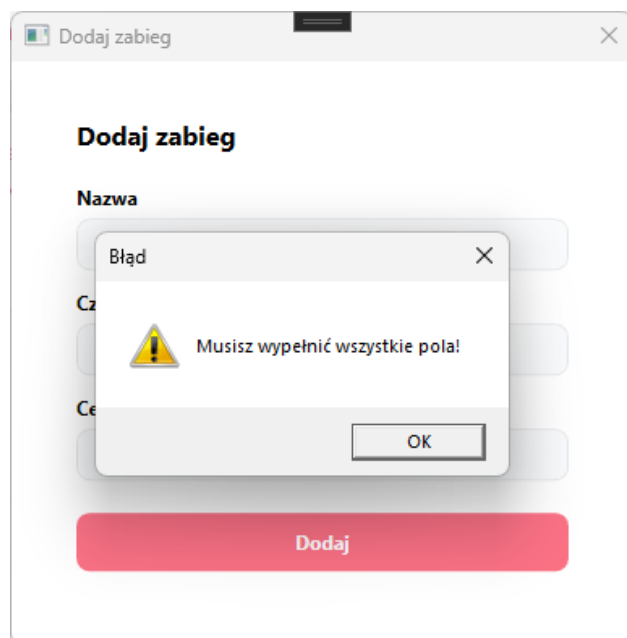
Rys. 5.42. Widok zakładki Zabiegi w panelu administratora.

Po kliknięciu przycisku (+ Dodaj zabieg) otwierane jest okno formularza zawierające trzy pola: nazwa zabiegu, czas trwania w minutach oraz cena w złotych.



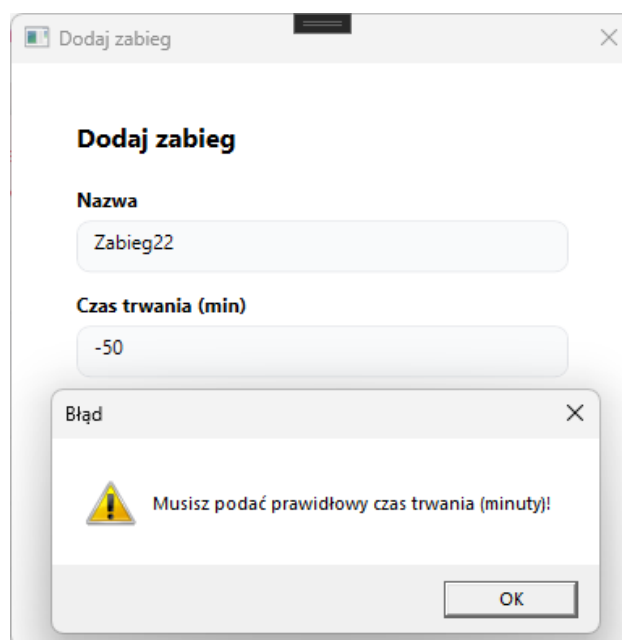
Rys. 5.43. Okno formularza dodawania zabiegu.

W przypadku gdy administrator nie wypełni wszystkich pól formularza, aplikacja wyświetla komunikat błędu.



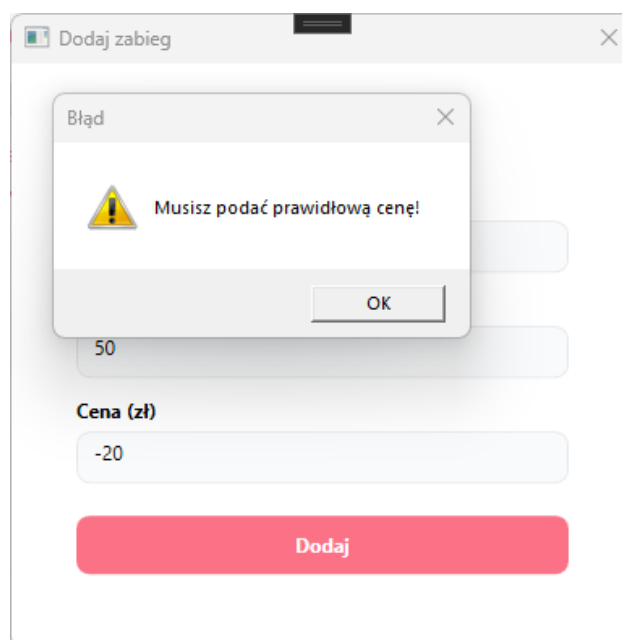
Rys. 5.44. Komunikat błędu przy niewypełnionych polach formularza.

Aplikacja waliduje również poprawność podanego czasu trwania tzn. musi być dodatnią liczbą całkowitą. W przypadku podania nieprawidłowej wartości wyświetlany jest komunikat błędu.



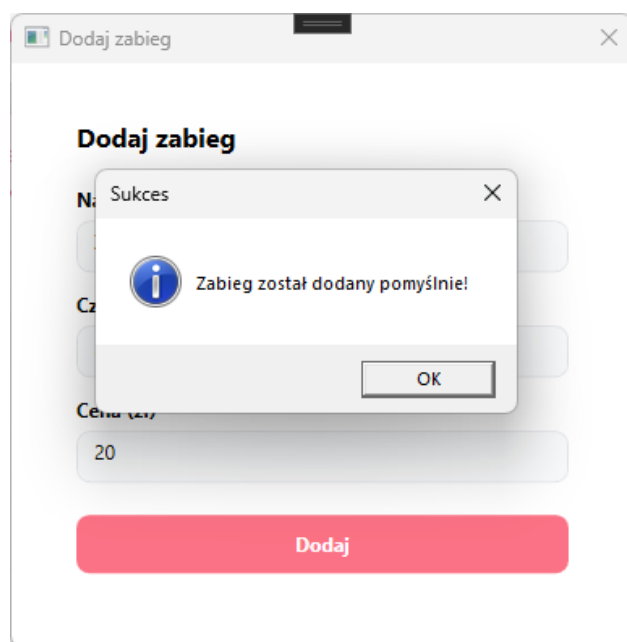
Rys. 5.45. Komunikat błędu przy nieprawidłowym czasie trwania zabiegu.

Analogicznie walidowana jest cena zabiegu tzn. nie może być ona liczbą ujemną. W przypadku podania nieprawidłowej wartości wyświetlany jest odpowiedni komunikat błędu.



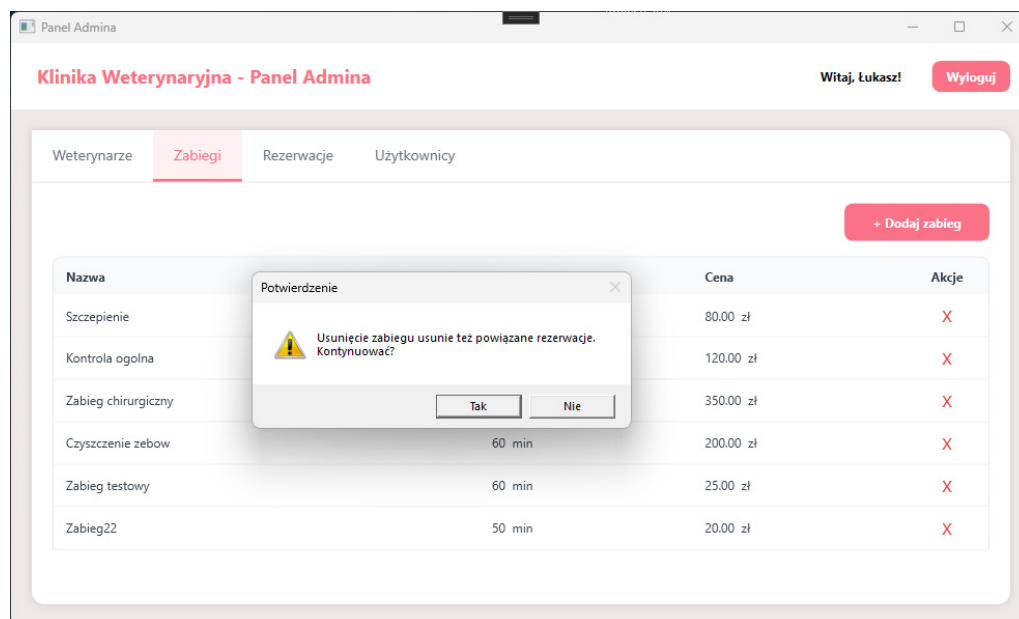
Rys. 5.46. Komunikat błędu przy nieprawidłowej cenie zabiegu.

Po poprawnym wypełnieniu wszystkich pól i kliknięciu przycisku Dodaj nowy zabieg zostaje zapisany w bazie danych, a aplikacja wyświetla komunikat potwierdzający pomyślne dodanie zabiegu.



Rys. 5.47. Komunikat potwierdzający pomyślne dodanie zabiegu.

Po kliknięciu przycisku X przy wybranym zabiegu aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie operacji, informując że usunięcie zabiegu spowoduje również usunięcie powiązanych rezerwacji. Po potwierdzeniu zabieg zostaje trwale usunięty z systemu.

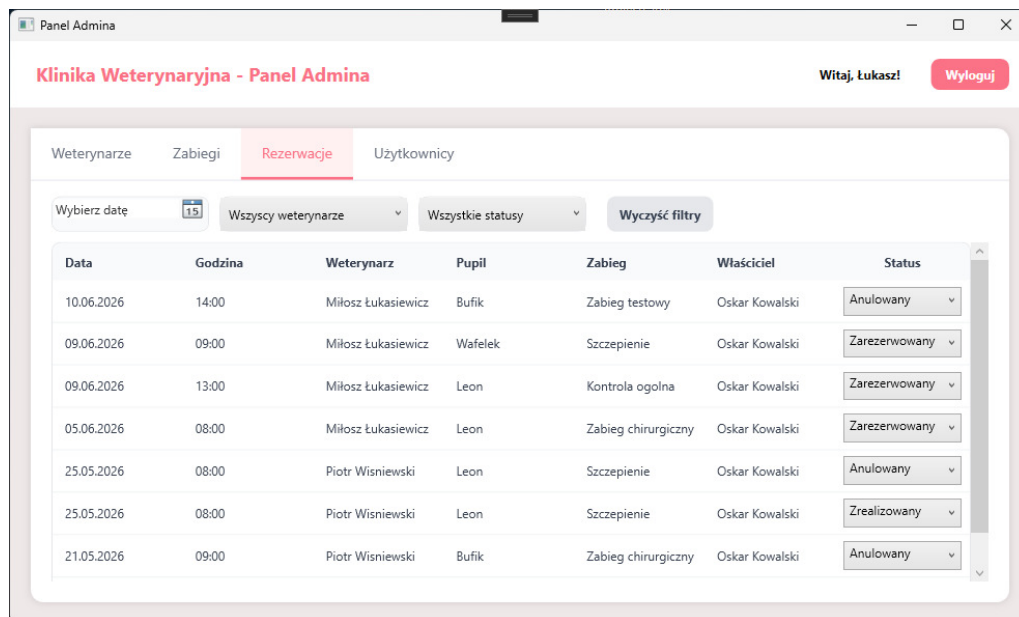


Rys. 5.48. Okno potwierdzenia usunięcia zabiegu.

Zakładka "Rezerwacje"

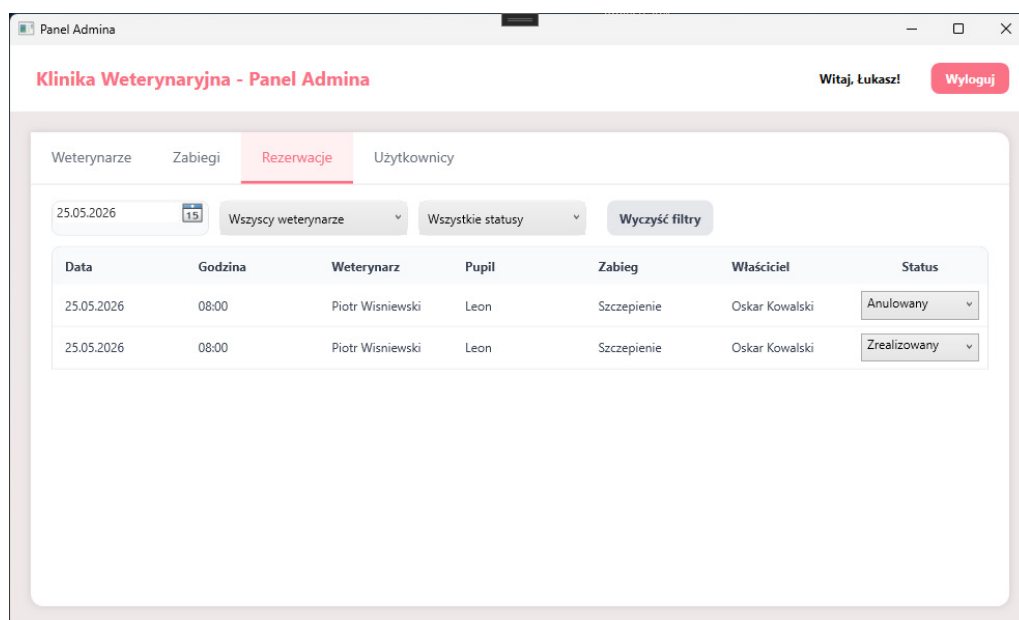
Zakładka "Rezerwacje" wyświetla listę wszystkich rezerwacji wizyt w systemie. Dla każdej rezerwacji prezentowane są informacje takie jak data, godzina, imię i nazwisko weterynarza, imię pupila, nazwa zabiegu, imię i nazwisko właściciela oraz aktualny status wizyty. Administrator może zmieniać status każdej rezerwacji bezpośrednio z poziomu listy przy użyciu listy rozwijanej z trzema opcjami: Zarezerwowany,

Zrealizowany oraz Anulowany. Nad listą dostępne są filtry: filtr daty, filtr weterynarza oraz filtr statusu, a także przycisk Wyczyść filtry resetujący wszystkie filtry do wartości domyślnych.



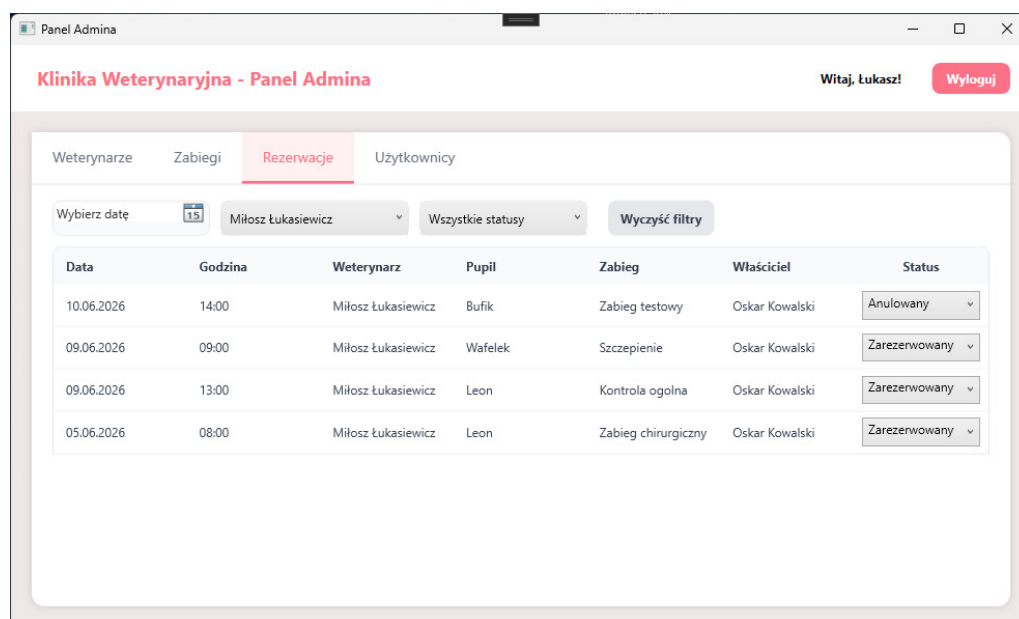
Rys. 5.49. Widok zakładki Rezerwacje w panelu administratora.

Filtr daty umożliwia wyświetlenie wyłącznie rezerwacji z wybranego dnia. Poniżej przedstawiono widok po zastosowaniu filtra daty na dzień 25.05.2026 czyli lista zawiera tylko rezerwacje z tego dnia.



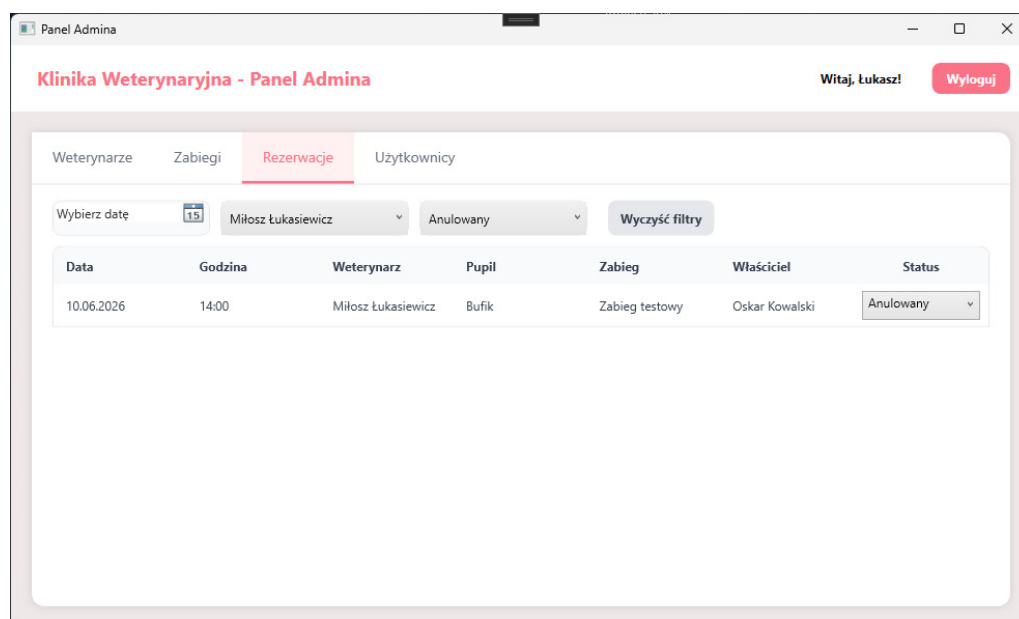
Rys. 5.50. Widok zakładki Rezerwacje po filtrowaniu po dacie.

Filtr weterynarza umożliwia wyświetlenie wyłącznie rezerwacji przypisanych do wybranego weterynarza. Poniżej przedstawiono widok po wybraniu weterynarza Miłoz Łukasiewicz czyli lista zawiera tylko jego rezerwacje.



Rys. 5.51. Widok zakładki Rezerwacje po filtrowaniu po weterynarzu.

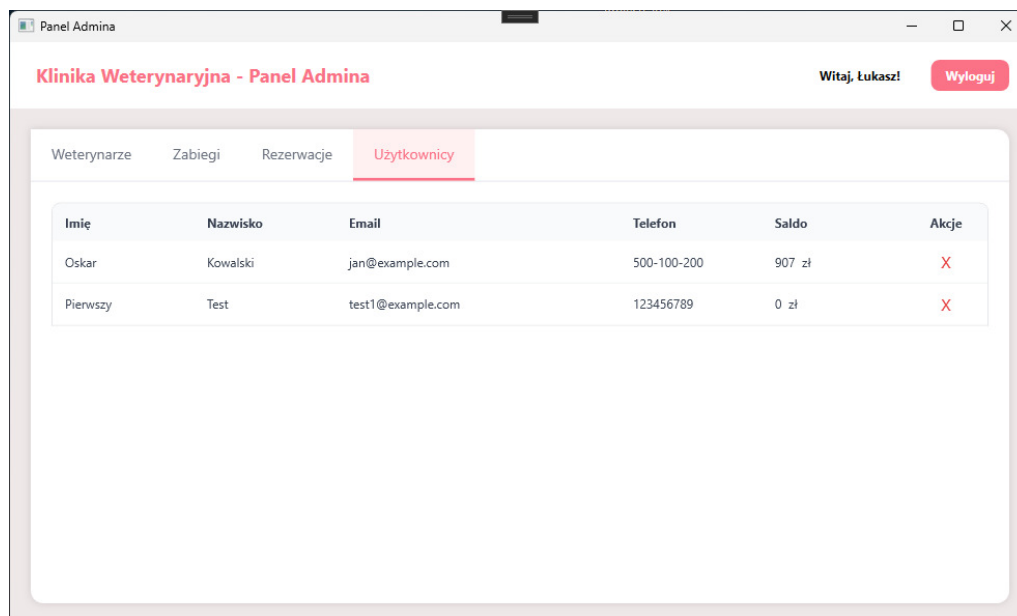
Filtry można łączyć, poniżej przedstawiono widok po jednoczesnym zastosowaniu filtra weterynarza (Miłosz Łukasiewicz) oraz filtra statusu (Anulowany), co pozwala precyzyjnie odnaleźć konkretne rezerwacje spełniające oba kryteria.



Rys. 5.52. Widok zakładki Rezerwacje po filtrowaniu po weterynarzu i statusie.

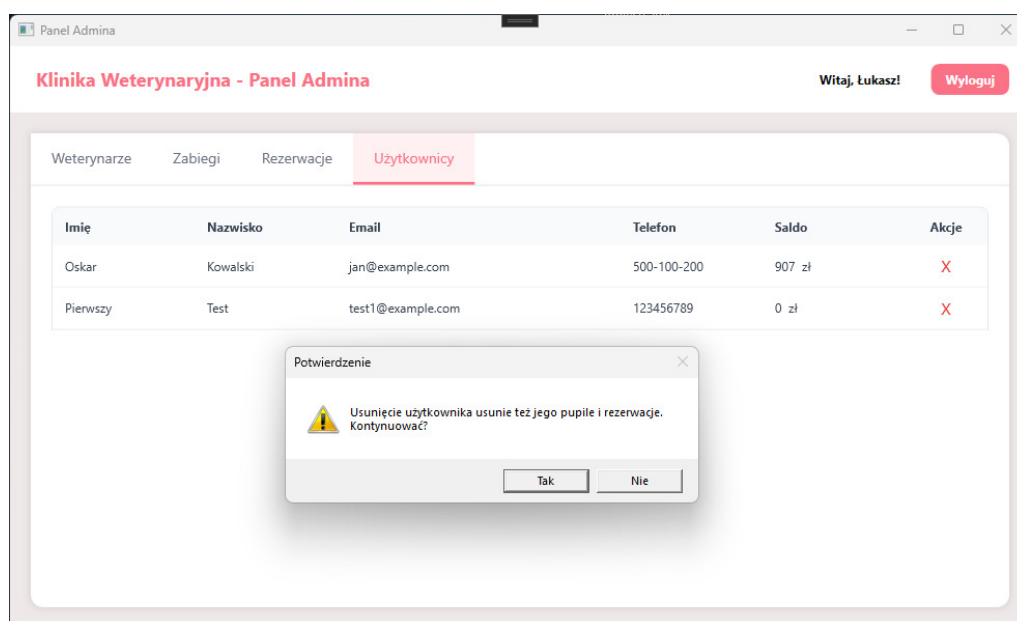
Zakładka "Użytkownicy"

Zakładka "Użytkownicy" wyświetla listę wszystkich zarejestrowanych użytkowników systemu z rolą Użytkownik. Dla każdego użytkownika prezentowane są informacje takie jak imię, nazwisko, adres e-mail, numer telefonu oraz aktualne saldo konta. Przy każdym użytkowniku dostępny jest przycisk X umożliwiający usunięcie użytkownika z systemu.



Rys. 5.53. Widok zakładki Użytkownicy w panelu administratora.

Po kliknięciu przycisku X przy wybranym użytkowniku aplikacja wyświetla okno dialogowe z prośbą o potwierdzenie operacji, informując że usunięcie użytkownika spowoduje również usunięcie jego pupili i rezerwacji. Po potwierdzeniu użytkownik zostaje trwale usunięty z systemu wraz ze wszystkimi powiązanymi danymi, a lista użytkowników jest automatycznie odświeżana.



Rys. 5.54. Okno potwierdzenia usunięcia użytkownika.

5.4. Uruchomienie aplikacji

1. Pobrać repozytorium projektu z GitHub: https://github.com/lukaszwarzocha/Programowanie_Objektowe2.git
2. Otworzyć plik Projekt_PO_KW.sln w Visual Studio.

3. Upewnić się że connection string w pliku Database.cs jest poprawny:

```
Server=localhost;Database=klinika_weterynaryjna;  
Trusted_Connection=True;TrustServerCertificate=True;
```

4. Przywrócić pakiety NuGet. W tym celu kliknąć prawym przyciskiem myszy na rozwiązanie w Eksploratorze rozwiązań, a następnie wybrać Menedżer pakietów NuGet → Przywróć pakiety NuGet.
5. Zbudować rozwiązanie: Kompilacja → Kompiluj rozwiązanie lub skrótem klawiszowym Ctrl+B.
6. Uruchomić aplikację klawiszem F5 lub przyciskiem Start.

Dane do logowania

W celu zalogowania się na konto administratora należy wykorzystać następujące dane:

- Login: admin@example.com
- Hasło: haslo123

W celu zalogowania się na konto użytkownika można wykorzystać następujące dane:

- Login: jan@example.com
- Hasło: haslo1234

6. Podsumowanie

Do prac zrealizowanych w aplikacji można zaliczyć przede wszystkim utworzenie panelu logowania, rejestracji, użytkownika i administratora z walidacją pól oraz obsługą błędów.

Panel użytkownika umożliwia:

- Wyświetlanie imienia oraz aktualnego salda konta,
- Przeglądanie i edycję danych osobowych oraz zmianę hasła,
- Dodawanie, edycję wieku pupila oraz ich usuwanie,
- Rezerwację wizyty w trzech krokach: wybór pupila i zabiegu, wybór weterynarza oraz wybór terminu z kalendarza dostępności,
- Weryfikację salda konta przed dokonaniem rezerwacji,
- Przeglądanie historii wizyt wraz z ich statusami,
- Anulowanie zarezerwowanych wizyt,
- Doładowanie salda konta,
- Opcję wylogowania.

Panel administratora umożliwia:

- Wyświetlanie imienia zalogowanego administratora,
- Dodawanie weterynarzy wraz z godzinami pracy oraz usuwanie ich z systemu,
- Zarządzanie zabiegami przypisanymi do weterynarzy,
- Dodawanie nowych zabiegów do oferty kliniki oraz ich usuwanie,
- Przeglądanie wszystkich rezerwacji z możliwością filtrowania po dacie, weterynarzu i statusie,
- Zmianę statusu rezerwacji,
- Automatyczną aktualizację statusu przeterminowanych rezerwacji przy uruchomieniu aplikacji,
- Przeglądanie wszystkich zarejestrowanych użytkowników wraz z ich saldem konta,
- Usuwanie użytkowników wraz z kaskadowym usunięciem ich pupila i rezerwacji,
- Opcję wylogowania.

Możliwymi pracami rozwojowymi w aplikacji będą między innymi:

- Szyfrowanie haseł w bazie danych,
- Dodanie panelu logowania dla weterynarzy z widokiem ich kalendarza wizyt,

- Powiadomienia e-mail lub SMS przypominające użytkownikom o zbliżających się wizytach,
- Rozszerzenie systemu o obsługę dodatkowych gatunków zwierząt,
- Implementacja systemu ocen i opinii weterynarzy przez użytkowników,
- Rozbudowa panelu administratora o liczne statystyki.

Bibliografia

- [1] Microsoft. Ado.net documentation, 2024. Accessed: 2026-05-05.
- [2] Microsoft. C# documentation, 2024. Accessed: 2026-05-04.
- [3] Microsoft. Install sql server management studio (ssms), 2024. Accessed: 2026-05-02.
- [4] Microsoft. Microsoft.data.sqlclient - microsoft learn, 2024. Accessed: 2026-05-03.
- [5] Microsoft. Xaml overview (wpf .net), 2024. Accessed: 2026-05-04.

Spis rysunków

3.1	Sterownik Microsoft.Data.SqlClient (Menadżer pakietów NuGet).	12
3.2	Struktura klas w projekcie.	15
4.1	Diagram Gantta.	38
5.1	Widok panelu logowania.	39
5.2	Komunikat błędu przy nieprawidłowych danych logowania.	40
5.3	Widok panelu rejestracji.	40
5.4	Komunikat błędu przy niewypełnionych polach formularza.	41
5.5	Komunikat błędu przy nieprawidłowym formacie numeru telefonu.	41
5.6	Komunikat błędu przy niezgodności haseł.	42
5.7	Komunikat błędu przy hasle krótszym niż 6 znaków.	42
5.8	Widok zakładki Moje konto.	43
5.9	Komunikat informacyjny przy braku zmian w danych.	43
5.10	Komunikat błędu przy niewypełnionych polach zmiany hasła.	44
5.11	Komunikat błędu przy nieprawidłowym obecnym hasle.	44
5.12	Komunikat błędu przy niezgodności nowych haseł.	45
5.13	Komunikat błędu przy hasle krótszym niż 6 znaków.	45
5.14	Widok zakładki Moje pupile.	46
5.15	Okno dialogowe edycji wieku pupila.	46
5.16	Komunikat błędu przy nieprawidłowym wieku pupila.	47
5.17	Okno potwierdzenia usunięcia pupila.	47
5.18	Widok zakładki Moje wizyty.	48
5.19	Okno potwierdzenia anulowania wizyty.	48
5.20	Widok okna Dodaj pupila.	49
5.21	Komunikat błędu przy niewypełnionych polach formularza.	50
5.22	Komunikat potwierdzający pomyślne dodanie pupila.	51
5.23	Krok 1 - widok wyboru pupila i zabiegu.	52
5.24	Komunikat błędu przy braku wybranego pupila.	52
5.25	Komunikat błędu przy braku wybranego zabiegu.	53
5.26	Krok 2 - widok wyboru weterynarza.	53
5.27	Komunikat błędu przy braku wybranego weterynarza.	54
5.28	Krok 3 - widok kalendarza dostępności weterynarza.	55

5.29	Komunikat błędu przy braku wybranego terminu.	55
5.30	Komunikat potwierdzający pomyślne zapisanie rezerwacji.	56
5.31	Widok zakładki Dodaj środki.	57
5.32	Komunikat błędu przy pustym polu kwoty.	57
5.33	Komunikat błędu przy podaniu nieprawidłowej kwoty.	58
5.34	Okno potwierdzenia wpłaty środków.	58
5.35	Komunikat potwierdzający pomyślną wpłatę środków.	59
5.36	Widok zakładki Weterynarze w panelu administratora.	59
5.37	Okno formularza dodawania weterynarza.	60
5.38	Komunikat błędu przy niewypełnionych wymaganych polach.	61
5.39	Okno zarządzania zabiegami weterynarza.	62
5.40	Komunikat potwierdzający aktualizację zabiegów weterynarza.	62
5.41	Okno potwierdzenia usunięcia weterynarza.	63
5.42	Widok zakładki Zabiegi w panelu administratora.	63
5.43	Okno formularza dodawania zabiegu.	64
5.44	Komunikat błędu przy niewypełnionych polach formularza.	64
5.45	Komunikat błędu przy nieprawidłowym czasie trwania zabiegu.	65
5.46	Komunikat błędu przy nieprawidłowej cenie zabiegu.	65
5.47	Komunikat potwierdzający pomyślne dodanie zabiegu.	66
5.48	Okno potwierdzenia usunięcia zabiegu.	66
5.49	Widok zakładki Rezerwacje w panelu administratora.	67
5.50	Widok zakładki Rezerwacje po filtrowaniu po dacie.	67
5.51	Widok zakładki Rezerwacje po filtrowaniu po weterynarzu.	68
5.52	Widok zakładki Rezerwacje po filtrowaniu po weterynarzu i statusie.	68
5.53	Widok zakładki Użytkownicy w panelu administratora.	69
5.54	Okno potwierdzenia usunięcia użytkownika.	69

Spis tabel

3.1	Diagram ERD projektowanej aplikacji.	13
-----	--	----

Spis listingów

3.1	Klasa Database	11
3.2	Metoda logowania wraz z SessionHelper oraz metodą z zapytaniem SQL	16
3.3	Metoda zmiany hasła wraz z metodą zapytania SQL	17
3.4	Metoda nawigacji do zmiany panelu wyświetlania	19
3.5	Metoda wpłaty środków wraz z metodą zapytania SQL	20
3.6	Metoda wyboru zabiegu oraz wyboru weterynarza	21
3.7	Metoda do rezerwacji (zachowania ciągłości między etapami) wraz z metodą zapytania SQL	22
3.8	Metoda wyświetlająca kalendarz wraz z metodą zapytania SQL	24
3.9	Metody usuwania i wczytywania użytkowników wraz z metodą zapytania SQL	28
3.10	Metoda dodawania weterynarza wraz z metodami zapytań SQL	30
3.11	Metody dodawania i usuwania zabiegów wraz z metodami zapytań SQL	32
3.12	Metoda aktualizacji statusów oraz metody związane z zabiegami wraz z metodami zapytań SQL	34

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....
Nazwa kierunku

.....
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: System zarządzania kliniką weterynaryjną
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

Rzeszów 7.06.2026
(miejscowość, data)

Lukasz Warzocha
(czytelny podpis studenta)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić